



BÁO CÁO ĐỒ ÁN HỆ THỐNG NHÚNG CHUYỂN ĐỔI MÃ BCD SANG LED 7 ĐOẠN

Giảng viên hướng dẫn:
TS. Trương Phong Tuyên

Sinh viên thực hiện:
Lê Tiến Đạt B2308281
Nguyễn Hoàng Khải B2308304



BÁO CÁO ĐỒ ÁN HỆ THỐNG NHÚNG CHUYỂN ĐỔI MÃ BCD SANG LED 7 ĐOẠN

Giảng viên hướng dẫn:
TS. Trương Phong Tuyên

Sinh viên thực hiện:
Lê Tiến Đạt B2308281
Nguyễn Hoàng Khải B2308304

◆ Đề tài: Chuyển mã BCD sang LED 7 đoạn

Yêu cầu: Giá trị BCD nhập từ SW, giá trị nhập hiển thị trên các LED đơn màu xanh, hiển thị kết quả ra LED 7 đoạn.

◆ 1. Tổng quan đề tài

1. Giới thiệu đề tài

- Trong lĩnh vực hệ thống nhúng, việc giao tiếp giữa vi xử lý và các ngoại vi (như công tắc, đèn LED) là bài toán cơ bản nhưng quan trọng.
- Đề tài hướng đến việc ứng dụng vi xử lý FPGA và phần mềm Quartus để thực hiện chương trình.



◆ 1. Tổng quan đề tài

2. Yêu cầu đề tài

- Đầu vào: Giá trị mã BCD nhập từ hệ thống công tắc trượt (SW).
- Đầu ra: Trạng thái nhập hiển thị trên LED đơn màu xanh (LEDG) và sử dụng LED đơn màu đỏ (LEDR) để quan sát trạng thái công tắc trượt (SW). Kết quả giải mã hiển thị ra LED 7 đoạn (HEX3_0).
- Phần cứng: Sử dụng phần mềm Quartus để thiết kế sơ đồ nguyên lý (Schematic) và cấu hình chân (Pin Planner) tương ứng với kit EP4CE115F29C7.
- Phần mềm: Sử dụng Nios II Eclipse được tích hợp sẵn trong Quartus để lập trình phần mềm bằng ngôn ngữ C.

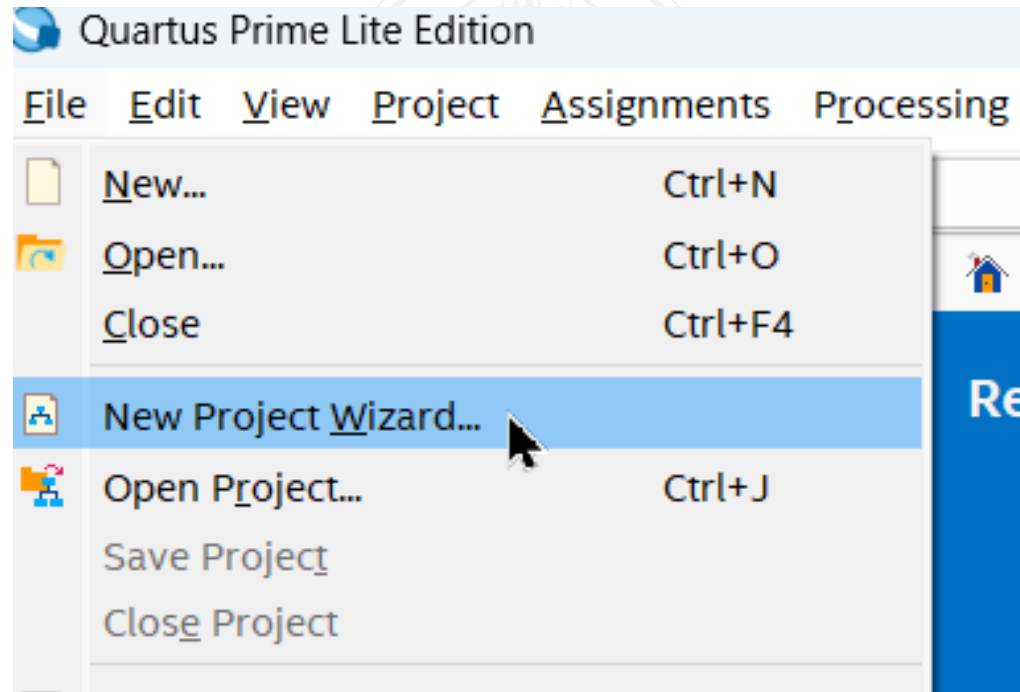
◆ 1. Tổng quan đề tài

3. Mục tiêu đề tài

- Xây dựng thành công hệ thống phần cứng hoạt động ổn định trên dòng chip Altera Cyclone IV E EP4CE115F29C7.
- Ứng dụng ngôn ngữ C để lập trình đọc/ghi dữ liệu qua các cổng PIO.
- Xử lý chính xác việc chuyển đổi mã BCD đầu vào từ công tắc (SW) sang tín hiệu hiển thị trên LED 7 đoạn và LED đơn.
- Nắm vững nguyên lý giao tiếp phần cứng - phần mềm qua các cổng ngoại vi (PIO).

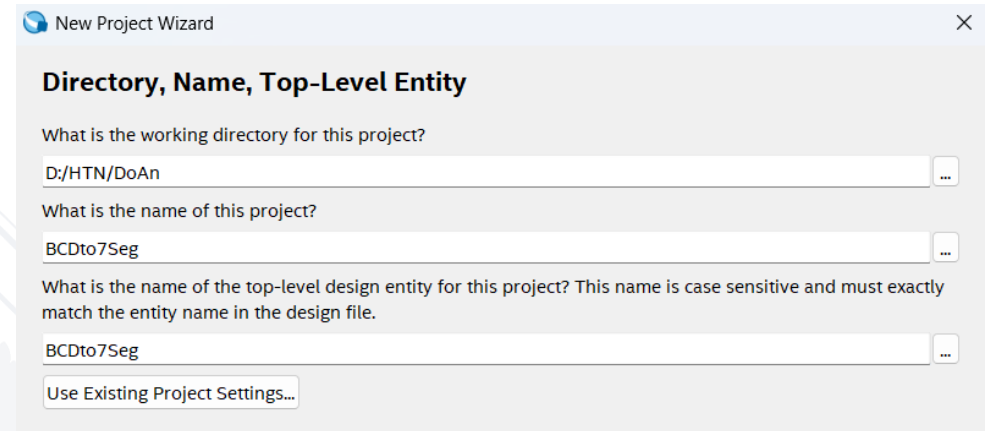
❖ 2. Tạo dự án thiết kế mới

- Tạo dự án mới vào *File - New Project Wizard..* trên thanh công cụ của cửa sổ Quartus.

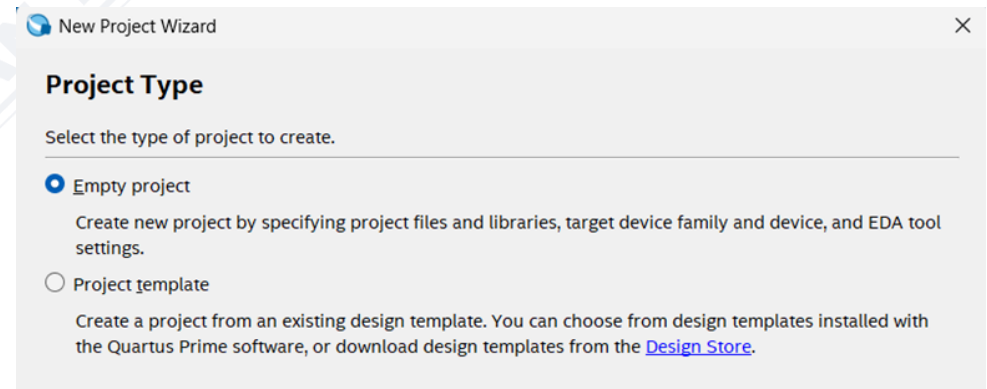


❖ 2. Tạo dự án thiết kế mới

- Ở cửa sổ *Directory, Name, Top-Level Entity*, chọn vị trí lưu trữ và đặt tên cho dự án. Sau đó chọn *Next* để tiếp tục.
- Ở cửa sổ *Project Type* chọn *Empty project*. Và chọn *Next*.



The screenshot shows the 'New Project Wizard' dialog box with the title 'New Project Wizard'. The main heading is 'Directory, Name, Top-Level Entity'. It contains three text input fields with browse buttons (three dots) to the right. The first field is labeled 'What is the working directory for this project?' and contains 'D:/HTN/DoAn'. The second field is labeled 'What is the name of this project?' and contains 'BCDto7Seg'. The third field is labeled 'What is the name of the top-level design entity for this project? This name is case sensitive and must exactly match the entity name in the design file.' and also contains 'BCDto7Seg'. At the bottom, there is a button labeled 'Use Existing Project Settings...'.



The screenshot shows the 'New Project Wizard' dialog box with the title 'New Project Wizard'. The main heading is 'Project Type'. It contains a label 'Select the type of project to create.' followed by two radio button options. The first option is 'Empty project', which is selected (indicated by a blue dot). Below it is a description: 'Create new project by specifying project files and libraries, target device family and device, and EDA tool settings.' The second option is 'Project template', which is not selected. Below it is a description: 'Create a project from an existing design template. You can choose from design templates installed with the Quartus Prime software, or download design templates from the [Design Store](#).'

❖ 2. Tạo dự án thiết kế mới

Ở cửa sổ *Family, Device & Board Settings* chọn dòng vi xử lý Cyclone IV E và chọn tên của thiết bị là *EP4CE115F29C7*, rồi chọn *Next*.

Family, Device & Board Settings

Device Board

Select the family and device you want to target for compilation.
You can install additional device support with the Install Devices command on the Tools menu.

To determine the version of the Quartus Prime software in which your target device is supported, refer to the [Device Support List](#) webpage.

Device family

Family: Cyclone IV E

Device: All

Target device

☐ Auto device selected by the Fitter

☒ Specific device selected in 'Available devices' list

☐ Other: n/a

Show in 'Available devices' list

Package: Any

Pin count: Any

Core speed grade: Any

Name filter: ep4ce115f29c7

☒ Show advanced devices

Available devices:

Name	Core Voltage	LEs	Total I/Os	GPIOs	Memory Bits	Embedded multiplier 9-bit element
EP4CE115F29C7	1.2V	114480	529	529	3981312	532

2. Tạo dự án thiết kế mới

- Ở cửa sổ *EDA Tool Settings* thiết lập như mặc định và chọn *Next*.
- Sau đó ở cửa sổ *Summary* chọn *Finish* để hoàn thành việc tạo dự án mới.

New Project Wizard

EDA Tool Settings

Specify the other EDA tools used with the Quartus Prime software to develop your project.

EDA tools:

Tool Type	Tool Name	Format(s)	Run Tool Automatically
Design Entry/S...	<None>	<None>	☐ Run this tool automatically to synthesize the current design
Simulation	<None>	<None>	☐ Run gate-level simulation automatically after compilation
Board-Level	Timing	<None>	
	Symbol	<None>	
	Signal Integrity	<None>	
	Boundary Scan	<None>	

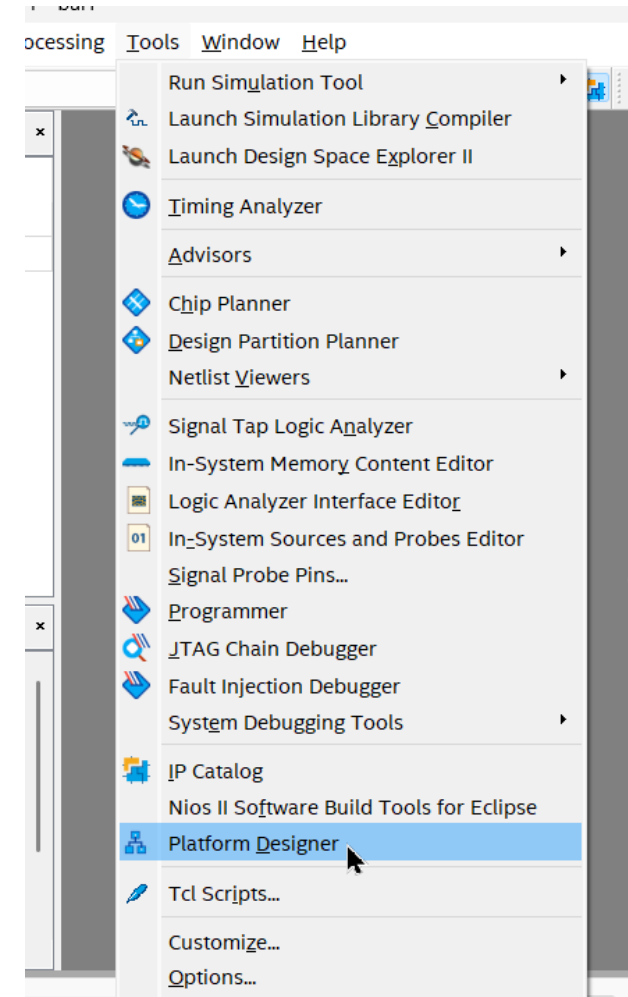
Summary

When you click Finish, the project will be created with the following settings:

Project directory:	D:/HTN/DoAn
Project name:	BCDto7Seg
Top-level design entity:	BCDto7Seg
Number of files added:	0
Number of user libraries added:	0
Device assignments:	
Design template:	n/a
Family name:	Cyclone IV E
Device:	EP4CE115F29C7
Board:	n/a
EDA tools:	
Design entry/synthesis:	<None> (<None>)
Simulation:	<None> (<None>)
Timing analysis:	()
Operating conditions:	
VCCINT voltage:	1.2V
Junction temperature range:	0-85 °C

❖ 3. Thiết kế hệ thống nhúng

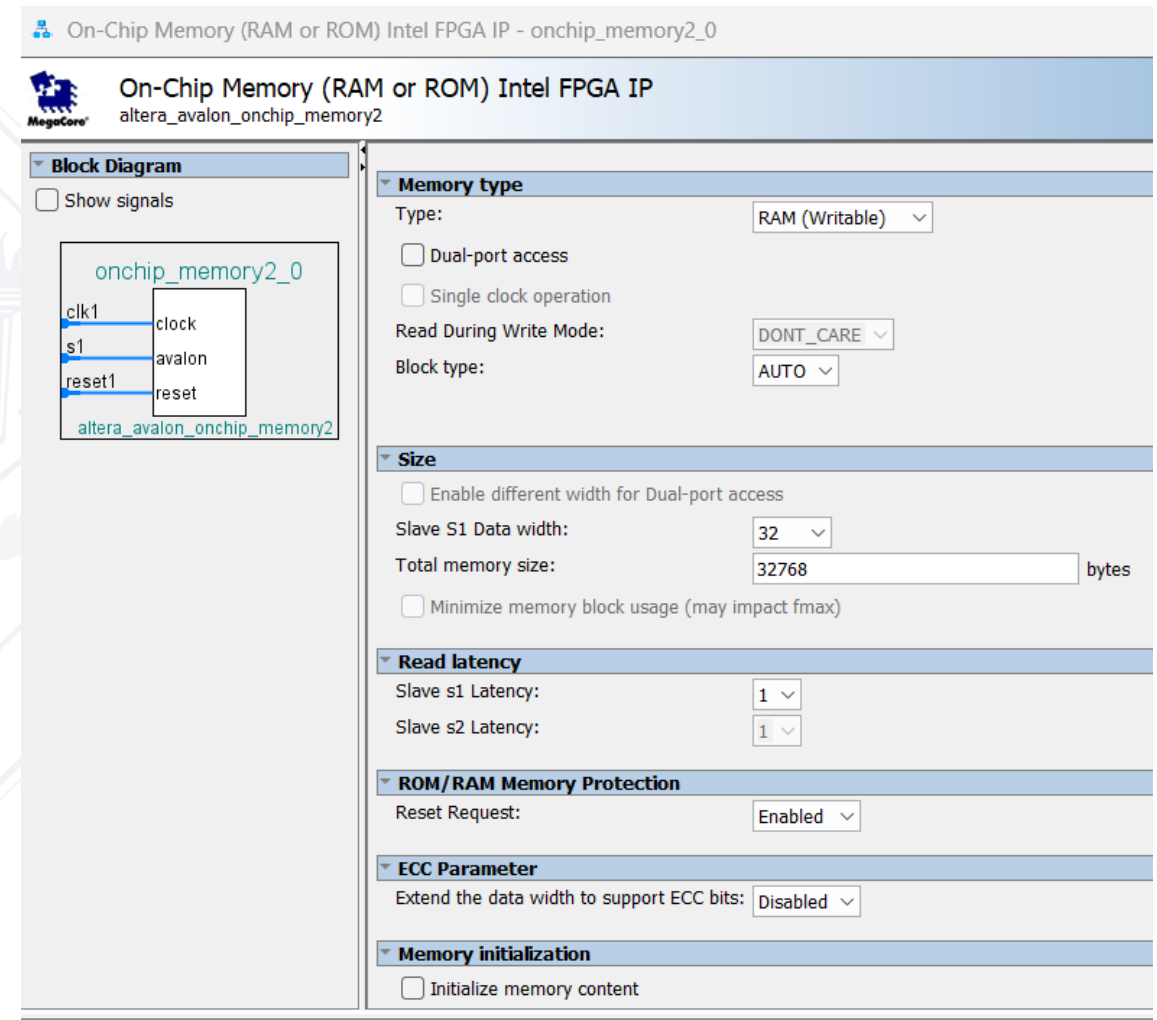
- Để tiến hành thiết kế hệ thống nhúng sử dụng các phần cứng sẵn có, trên thanh công cụ vào *Tools – Platform Designer*.
- Ta vào công cụ *Platform Designer* để cấu hình phần cứng và các ngoại vi PIO.



3. Thiết kế hệ thống nhúng

1. Bộ nhớ (RAM)

- Trên thanh cửa sổ *IP Catalog* tìm và chọn *On-Chip Memory (RAM or ROM) Intel FPGA IP*. Sau đó thiết lập cấu hình bộ nhớ như hình và chọn *Finish*.



3. Thiết kế hệ thống nhúng

2. Bộ xử lý (CPU)

- Trên thanh cửa sổ *IP Catalog* tìm và chọn *Nios II Processor*. Sau đó chọn bộ xử lý *Nios II/e* và chọn *Finish*.

Nios II Processor - nios2_gen2_0

Nios II Processor
altera_nios2_gen2

Block Diagram

Show signals

nios2_gen2_0

clk reset irq debug_mem_slave

clock reset interrupt avalon

data master instruction master debug_reset_request custom instruction master

altera_nios2_gen2

Main Vectors Caches and Memory Interfaces Arithmetic Instructions MMU and MPU Settings JTAG

Select an Implementation

Nios II Core: ☒ Nios II/e ☐ Nios II/f

	Nios II/e	Nios II/f
Summary	Resource-optimized 32-bit RISC	Performance-optimized 32-bit RISC
Features	JTAG Debug ECC RAM Protection	JTAG Debug Hardware Multiply/Divide Instruction/Data Caches Tightly-Coupled Masters ECC RAM Protection External Interrupt Controller Shadow Register Sets MPU MMU
RAM Usage	2 + Options	2 + Options

❖ 3. Thiết kế hệ thống nhúng

3. Công tắc trượt 2 vị trí (SWs)

- Để giao tiếp với 4 công tắc trượt thì trong cửa sổ *IP Catalog* tìm và chọn *PIO (Parallel I/O) Intel FPGA IP*.
- Sau đó cấu hình như hình và chọn *Finish*.

The screenshot shows the configuration window for the PIO (Parallel I/O) Intel FPGA IP. The window is titled "PIO (Parallel I/O) Intel FPGA IP" and "altera_avalon_pio".

Block Diagram: The block diagram shows the "pio_0" block with four inputs: "clk", "reset", "s1", and "external connection". These inputs are connected to the "clock", "reset", "avalon", and "conduit" pins of the "pio_0" block. The "pio_0" block is labeled "altera_avalon_pio".

Basic Settings:

- Width (1-32 bits): 4
- Direction: ☒ Input, ☐ Bidir, ☐ InOut, ☐ Output
- Output Port Reset Value: 0x0000000000000000

Output Register:

- ☐ Enable individual bit setting/clearing

Edge capture register:

- ☐ Synchronously capture
- Edge Type: RISING
- ☐ Enable bit-clearing for edge capture register

Interrupt:

- ☐ Generate IRQ
- IRQ Type: LEVEL
- Level:** Interrupt CPU when any unmasked I/O pin is logic true
- Edge:** Interrupt CPU when any unmasked bit in the edge-capture register is logic true. Available when synchronous capture is enabled

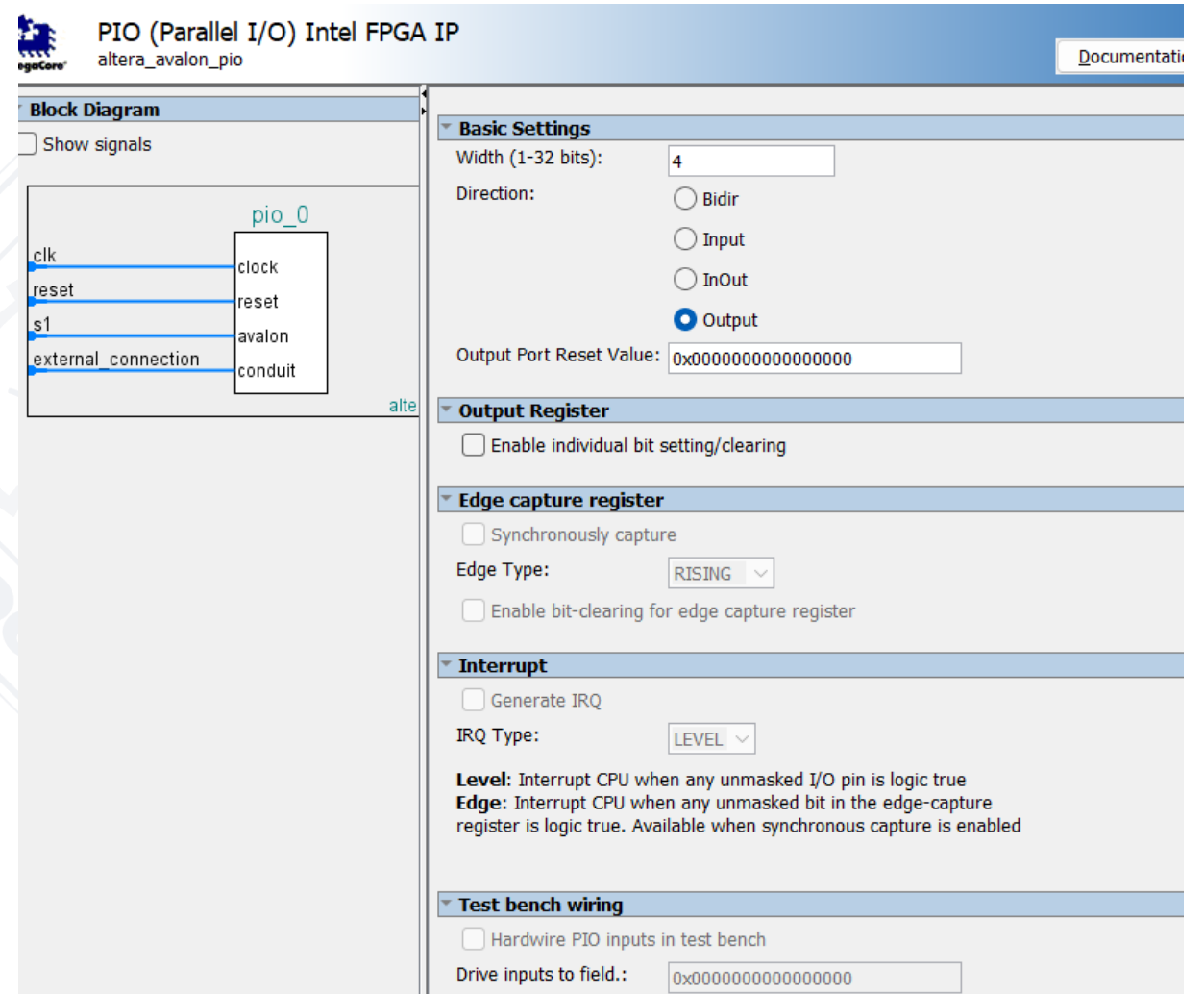
Test bench wiring:

- ☐ Hardwire PIO inputs in test bench
- Drive inputs to field.: 0x0000000000000000

❖ 3. Thiết kế hệ thống nhúng

4. LED đơn màu đỏ (LEDR)

- Vào *PIO* tiến hành thiết lập kết nối với 4 LED đơn đỏ giống với công tắc trượt (SWs) phía trên.
- Sau đó cấu hình như hình và chọn *Finish*.



❖ 3. Thiết kế hệ thống nhúng

5. LED đơn màu xanh (LEDG)

- Vào *PIO* tiến hành thiết lập kết nối với 4 LED đơn xanh giống với LED đơn đỏ (LEDR) phía trên.
- Sau đó cấu hình như hình và chọn *Finish*.

PIO (Parallel I/O) Intel FPGA IP
altera_avalon_pio

Documentation

Block Diagram

☐ Show signals

pio_0

clk → clock
reset → reset
s1 → avalon
external_connection → conduit

Basic Settings

Width (1-32 bits): 4

Direction: ☐ Bidir
☐ Input
☒ InOut
☒ Output

Output Port Reset Value: 0x0000000000000000

Output Register

☐ Enable individual bit setting/clearing

Edge capture register

☐ Synchronously capture

Edge Type: RISING

☐ Enable bit-clearing for edge capture register

Interrupt

☐ Generate IRQ

IRQ Type: LEVEL

Level: Interrupt CPU when any unmasked I/O pin is logic true
Edge: Interrupt CPU when any unmasked bit in the edge-capture register is logic true. Available when synchronous capture is enabled

Test bench wiring

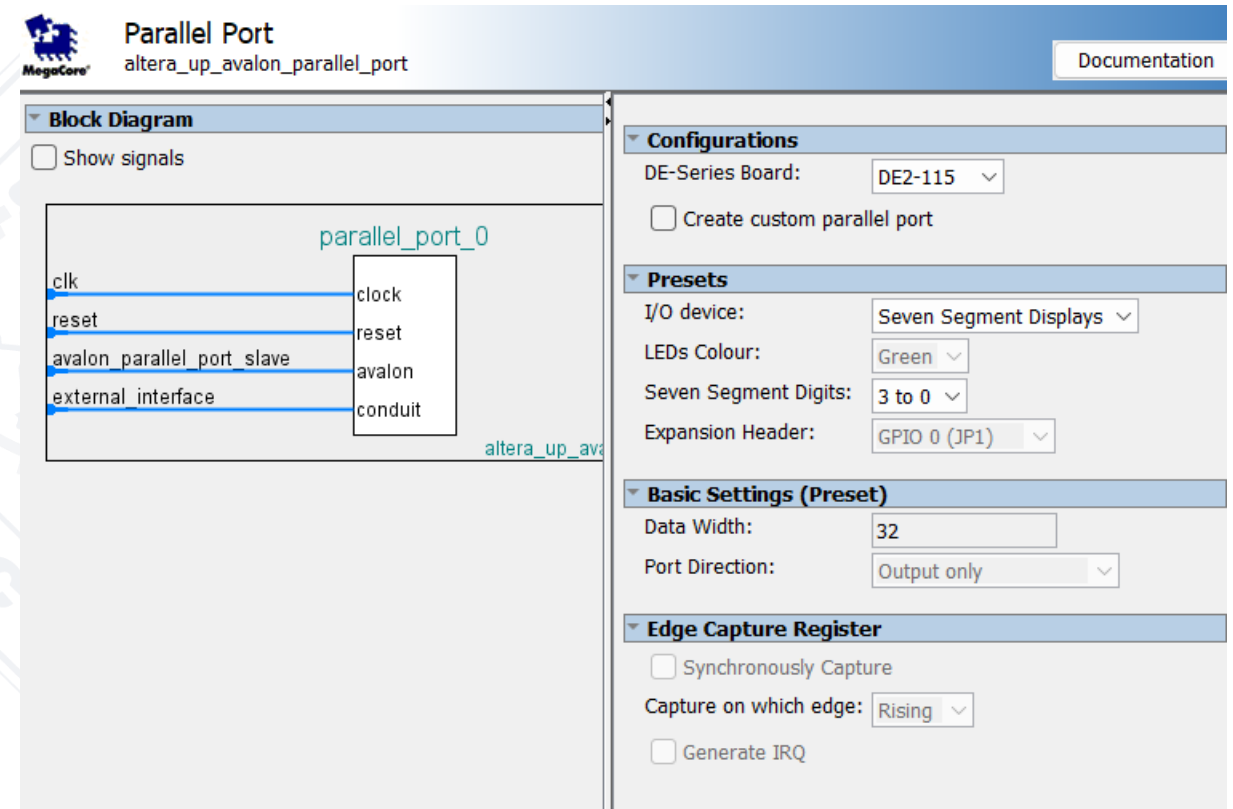
☐ Hardwire PIO inputs in test bench

Drive inputs to field.: 0x0000000000000000

❖ 3. Thiết kế hệ thống nhúng

6. LED 7 đoạn (HEX3_0)

- Để giao tiếp với LED 7 đoạn, trên cửa sổ *IP Catalog* tìm và chọn *Parallel Port*.
- Ở mục *I/O device* chọn *Seven Segment Displays*.
- Ở mục *Seven Segment Digits* chọn *3 to 0*.
- Sau đó chọn *Finish*.



3. Thiết kế hệ thống nhúng

7. Đổi tên các thành phần hệ thống

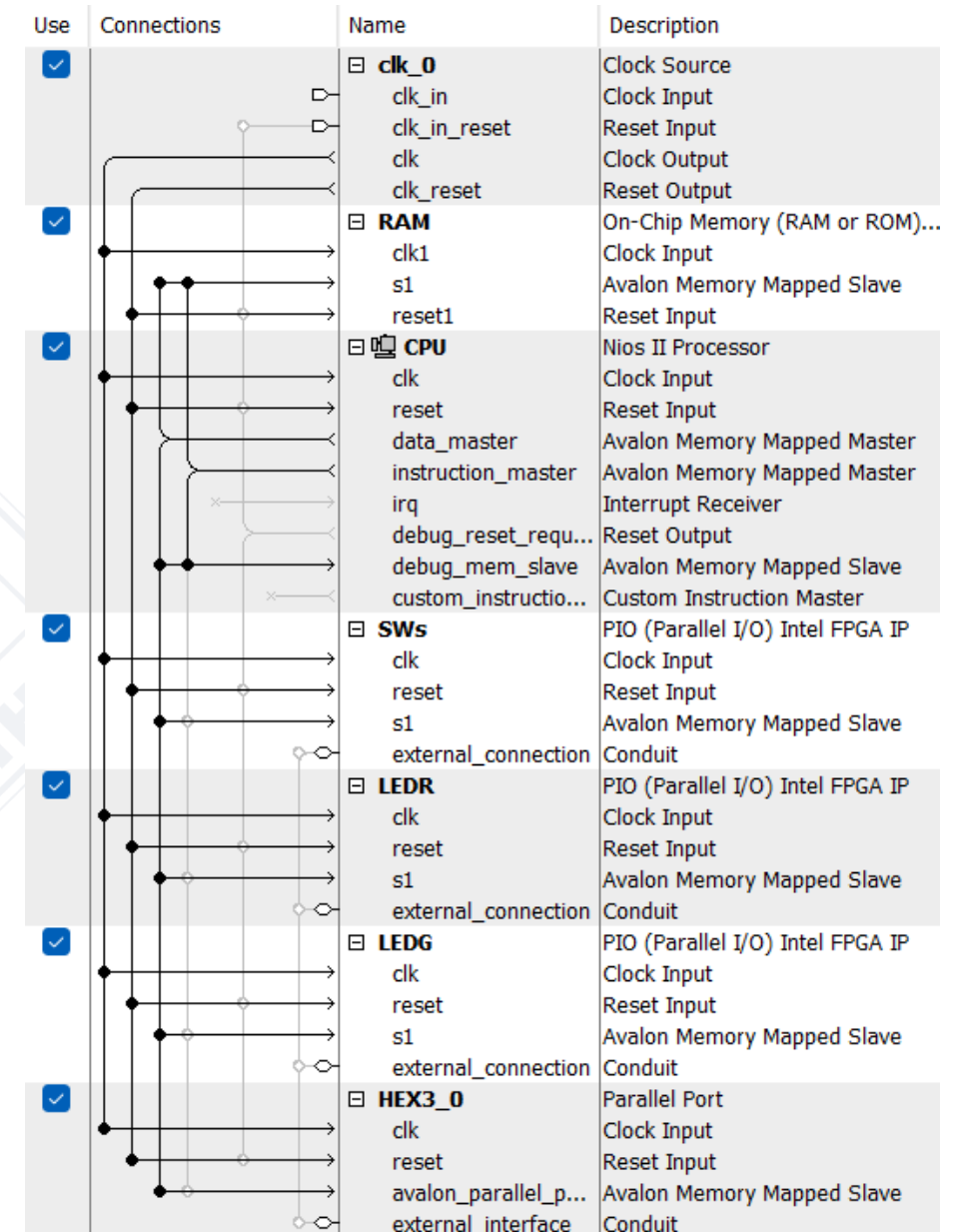
- Ở cửa sổ *System Content*, tiến hành đổi tên các phần cứng và ngoại vi để dễ quản lý.
- Có thể ấn tổ hợp phím *Ctrl + R* hoặc *Chuột phải - Rename* để đổi tên.

Use	Connections	Name	Description
☒		clk_0 - clk_in - clk_in_reset - clk - clk_reset	Clock Source - Clock Input - Reset Input - Clock Output - Reset Output
☒		RAM - clk1 - s1 - reset1	On-Chip Memory (RAM or ROM)... - Clock Input - Avalon Memory Mapped Slave - Reset Input
☒		CPU - clk - reset - data_master - instruction_master - irq - debug_reset_requ... - debug_mem_slave - custom_instructio...	Nios II Processor - Clock Input - Reset Input - Avalon Memory Mapped Master - Avalon Memory Mapped Master - Interrupt Receiver - Reset Output - Avalon Memory Mapped Slave - Custom Instruction Master
☒		SWs - clk - reset - s1 - external_connection	PIO (Parallel I/O) Intel FPGA IP - Clock Input - Reset Input - Avalon Memory Mapped Slave - Conduit
☒		LEDR - clk - reset - s1 - external_connection	PIO (Parallel I/O) Intel FPGA IP - Clock Input - Reset Input - Avalon Memory Mapped Slave - Conduit
☒		LEDG - clk - reset - s1 - external_connection	PIO (Parallel I/O) Intel FPGA IP - Clock Input - Reset Input - Avalon Memory Mapped Slave - Conduit
☒		HEX3_0 - clk - reset - avalon_parallel_p... - external_interface	Parallel Port - Clock Input - Reset Input - Avalon Memory Mapped Slave - Conduit

3. Thiết kế hệ thống nhúng

8. Kết nối các thành phần hệ thống

- Ở cửa sổ *System Content*, tiến hành kết nối các thành phần hệ thống.
- Có thể nhấn *Chuột trái* vào các dấu chấm ở cột *Connections* hoặc *Chuột phải* vào thành phần và chọn *Connections* tới từng thành phần khác.
- Ta đổi tên như hình sau đó ấn tổ hợp phím *Ctrl + S* để lưu.



◆ 3. Thiết kế hệ thống nhúng

9. Thiết lập bộ nhớ hoạt động của CPU

- Sau khi kết nối các thành phần hệ thống, ta *Click đúp chuột trái vào CPU* và chuyển sang tab *Vectors* để gán bộ nhớ hoạt động cho *CPU*:
 - Reset vector memory: RAM.s1
 - Exception vector memory: Ram.s1
- Sau đó ấn *Finish* để hoàn thành.

System: BCDto7Seg Path: CPU

Nios II Processor

altera_nios2_gen2 [Details](#)

Main Vectors Caches and Memory Interfaces Arithmetic Instructions MMU

▼ **Reset Vector**

Reset vector memory: RAM.s1

Reset vector offset: 0x00000000

Reset vector: 0x00008000

▼ **Exception Vector**

Exception vector memory: RAM.s1

Exception vector offset: 0x00000020

Exception vector: 0x00008020

▼ **Fast TLB Miss Exception Vector**

Fast TLB Miss Exception vector memory: None

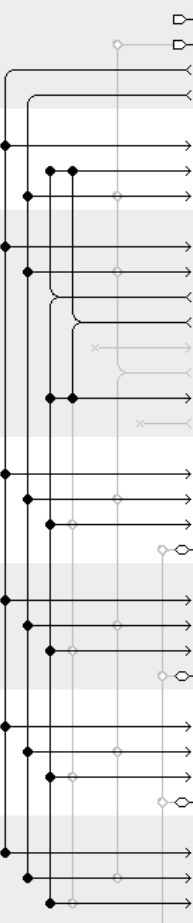
Fast TLB Miss Exception vector offset: 0x00000000

Fast TLB Miss Exception vector: 0x00000000

3. Thiết kế hệ thống nhúng

10. Gán địa chỉ nền cho các thành phần và Export kết nối với các ngoại vi

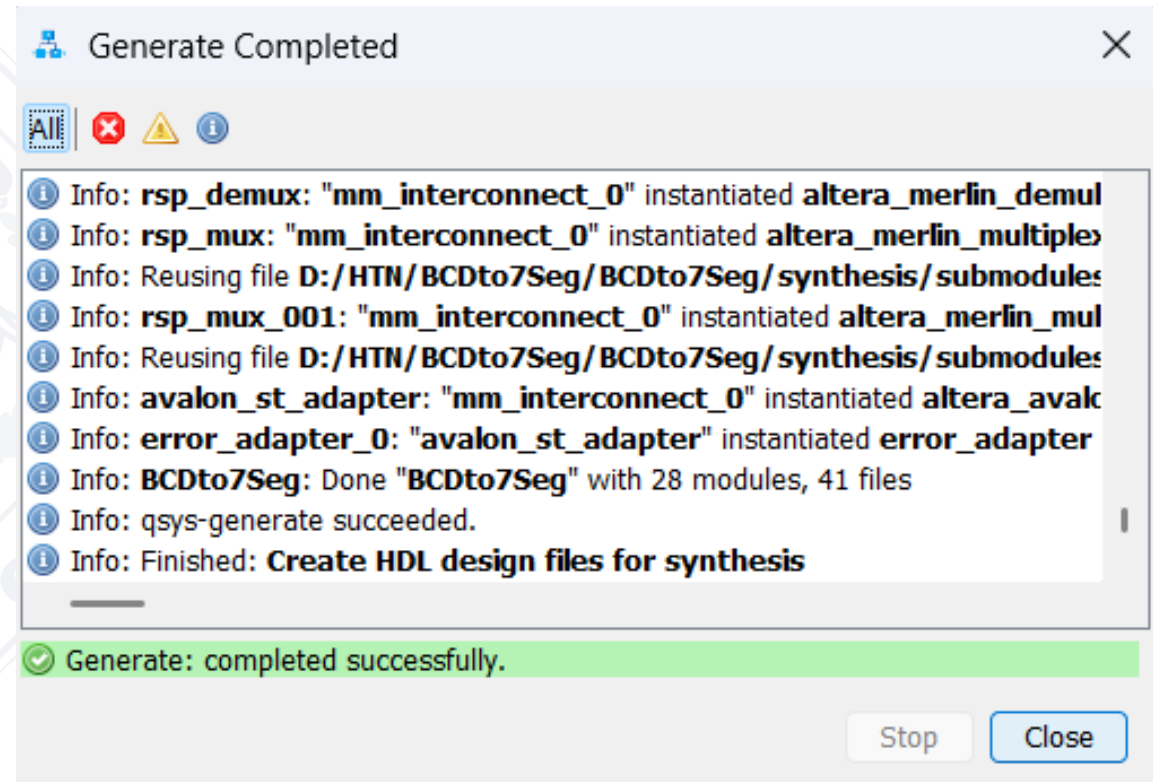
- Trên thanh công cụ, vào *System* chọn *Assign Base Addresses* để thực hiện gán chân tự động cho các thành phần hệ thống.
- Ở cửa sổ *System Contents*, trên cột *Description* tìm dòng *external_connection* (Conduit) và Click đúp chuột trái vào cột *Export* để export các kết nối.

Use	Connections	Name	Description	Export	Clock	Base	End
☒		clk_0 clk_in clk_in_reset clk clk_reset	Clock Source Clock Input Reset Input Clock Output Reset Output	clk reset Double-click to export Double-click to export	exported		
☒		RAM clk1 s1 reset1	On-Chip Memory (RAM or ROM)... Clock Input Avalon Memory Mapped Slave Reset Input	Double-click to export Double-click to export Double-click to export	clk_0 [clk1] [clk1]	0x0000_8000	0x0000_ffff
☒		CPU clk reset data_master instruction_master irq debug_reset_requ... debug_mem_slave custom_instructio...	Nios II Processor Clock Input Reset Input Avalon Memory Mapped Master Avalon Memory Mapped Master Interrupt Receiver Reset Output Avalon Memory Mapped Slave Custom Instruction Master	Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk] [clk] [clk] [clk]		IRQ 0 I
☒		SWs clk reset s1 external_connection	PIO (Parallel I/O) Intel FPGA IP Clock Input Reset Input Avalon Memory Mapped Slave Conduit	Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk]	0x0001_1020	0x0001_102f
☒		LEDR clk reset s1 external_connection	PIO (Parallel I/O) Intel FPGA IP Clock Input Reset Input Avalon Memory Mapped Slave Conduit	Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk]	0x0001_1010	0x0001_101f
☒		LEDG clk reset s1 external_connection	PIO (Parallel I/O) Intel FPGA IP Clock Input Reset Input Avalon Memory Mapped Slave Conduit	Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk]	0x0001_1000	0x0001_100f
☒		HEX3_0 clk reset avalon_parallel_p... external_interface	Parallel Port Clock Input Reset Input Avalon Memory Mapped Slave Conduit	Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk]	0x0001_1030	0x0001_103f

◆ 3. Thiết kế hệ thống nhúng

11. Lưu hệ thống và tạo tập tin HDL

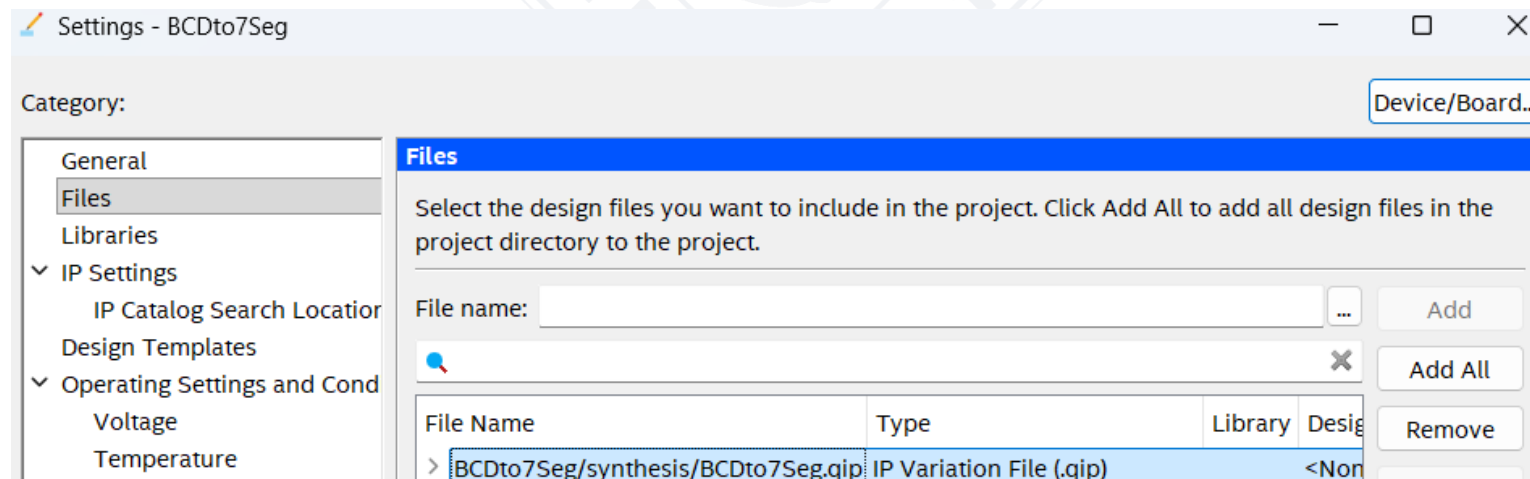
- Nhấn tổ hợp phím *Ctrl + S* hoặc trên thanh công cụ vào *File - Save* và đặt lưu lại với tên *BCDto7Seg*.
- Sau khi lưu xong, trên thanh công cụ vào *Generate - Generate HDL* để tạo tập tin HDL.
- Nếu xuất hiện lỗi (error) hoặc cảnh báo (warning) thì phải xem xét và sửa chữa cho đến khi tạo thành công tập tin HDL.
- Kết quả tạo tập tin thành công như hình bên.



◆ 4. Thiết kế sơ đồ nguyên lý và gán chân hệ thống

1. Thiết kế sơ đồ nguyên lý

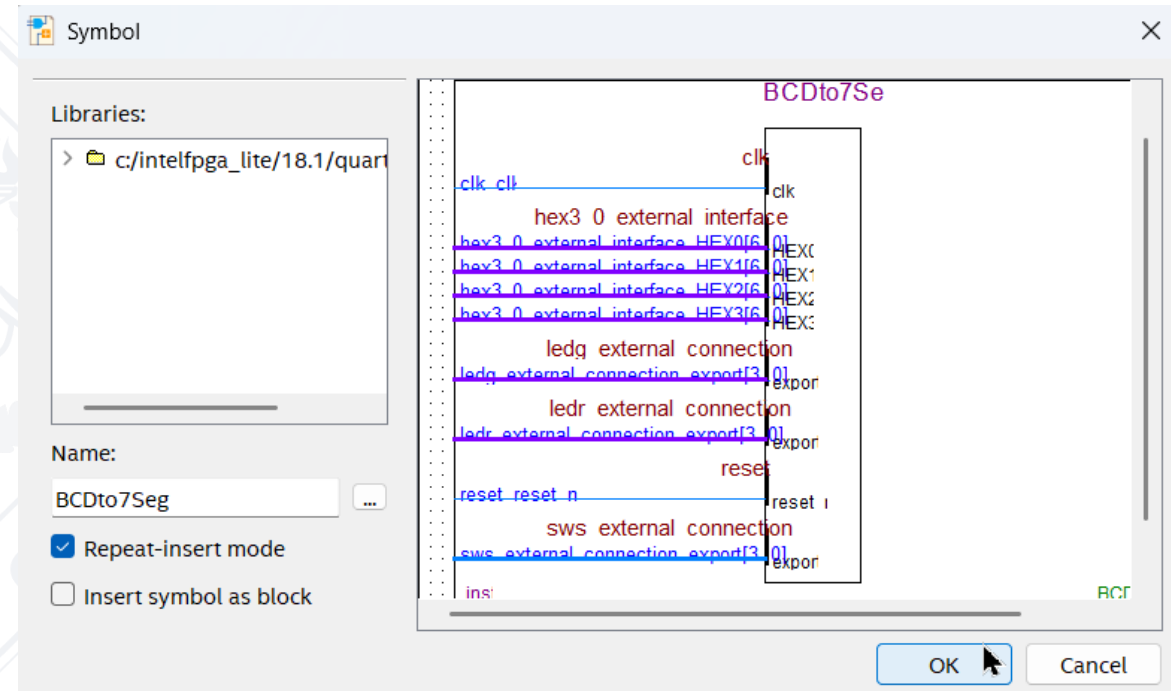
- Trong phần mềm *Quartus*, ở cửa *Project Navigator* chọn *Files*.
- Sau đó, *Click chuột phải* vào *Files* chọn *Add/Remove Files in Project...*
- Ở cửa sổ *Setting*, vào *Category - Files* chọn vào dấu ... ở sau textbox *File name* để thêm tập tin vào dự án.
- Chọn file có đuôi file là *.qip* thường nằm ở đường dẫn “ten_project/synthesis”



◆ 4. Thiết kế sơ đồ nguyên lý và gán chân hệ thống

1. Thiết kế sơ đồ nguyên lý

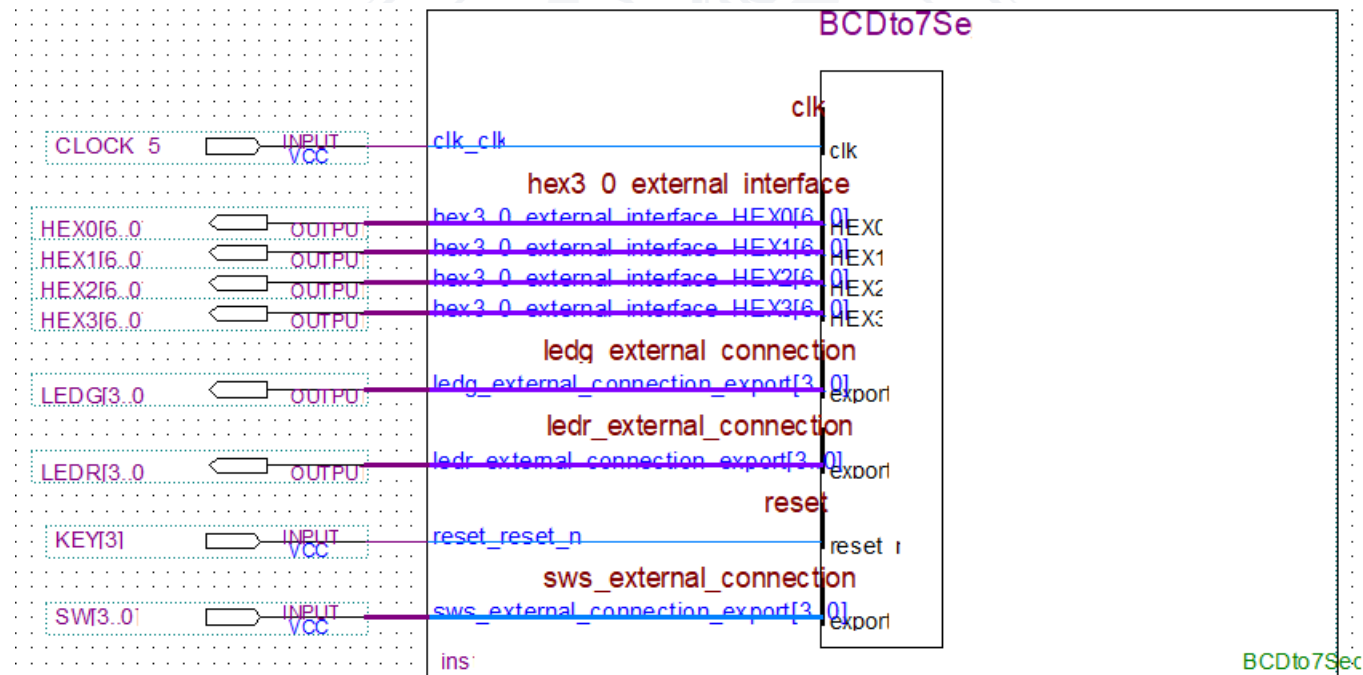
- Tiếp theo trên thanh công cụ vào *File - New* hoặc nhấn tổ hợp phím *Ctrl + N*. Chọn *Block Diagram/Schematic File* để tiến hành thiết kế sơ đồ nguyên lý. Lưu lại với tên *TestBCDto7Seg.bdf*
- Nhấp vào biểu tượng  để mở cửa sổ *Symbol Tool*. Tiếp theo Click chuột vào dấu ... và chọn file *BCDto7Seg.bsf* và chọn *Open* để thêm vào sơ đồ nguyên lý.
- Sau đó ở cửa sổ *Symbol* chọn *OK* để hoàn tất.



4. Thiết kế sơ đồ nguyên lý và gán chân hệ thống

1. Thiết kế sơ đồ nguyên lý

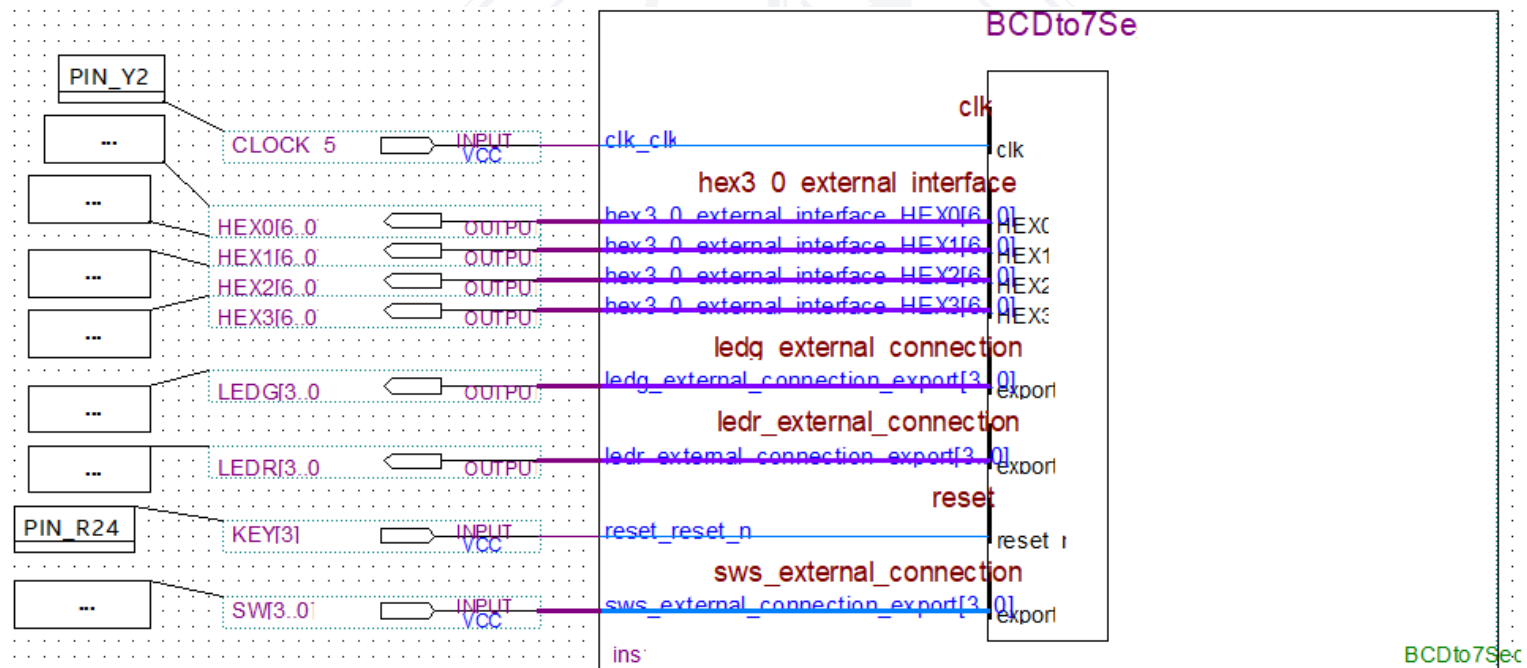
- Tiếp theo, có thể nối dây thủ công bằng *Pin Tool*  hoặc nối dây tự động bằng cách *Chuột phải* vào khối và chọn *Generate Pins for Symbol Ports*.
- Sau đó đổi tên các *Input/Output* tương ứng như hình bên và chọn *File* vừa thiết kế và chọn *Set as Top-Level Entity*.



◆ 4. Thiết kế sơ đồ nguyên lý và gán chân hệ thống

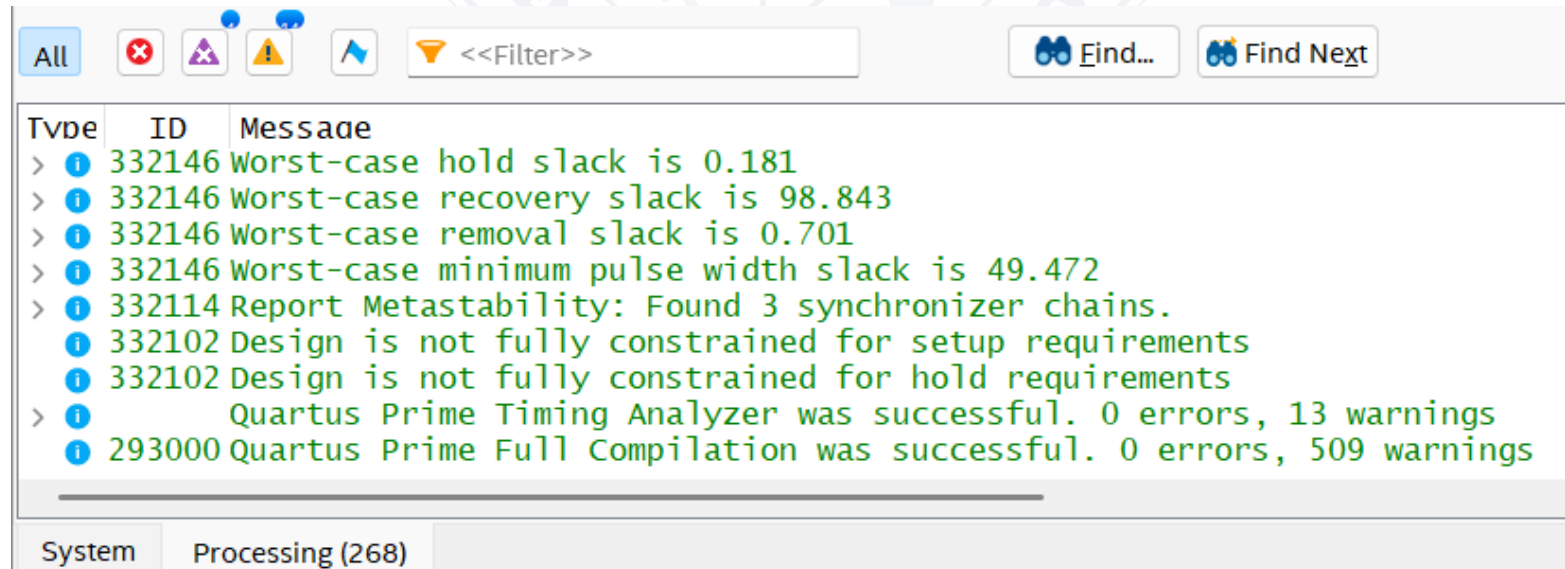
2. Gán chân cho hệ thống nhúng

- Trên thanh công cụ vào *Assignments - Import Assignments*.
- Chọn vào đường dẫn chứa tập tin *DE2_115_pin_assignments.qsf* hoặc *DE2_115_pin_assignments.cvs*, xong chọn OK để hoàn tất gán chân.
- Kết quả gán chân sẽ giống như hình bên dưới.



◆ 5. Biên dịch và kiểm tra thiết kế

- Trên thanh công cụ vào *Processing - Start Compilation* hoặc có thể ấn tổ hợp phím *Ctrl + L* để tiến hành biên dịch.
- Nếu kết quả biên dịch có cảnh báo (warning) hoặc lỗi (error) thì phải kiểm tra lại xem cảnh báo có ảnh hưởng gì đến chương trình không, nếu không thì có thể bỏ qua, còn lỗi thì phải sửa cho đến khi biên dịch thành công.
- Kết quả biên dịch thành công.



5. Biên dịch và kiểm tra thiết kế

- Để kiểm tra kết quả gán chân sau khi biên dịch, trên thanh công cụ vào *Assignments - Pin Planner* và nhấn vào tab *Groups* để kiểm tra gán chân cho từng nhóm.
- Kết quả gán chân hệ thống.

Groups

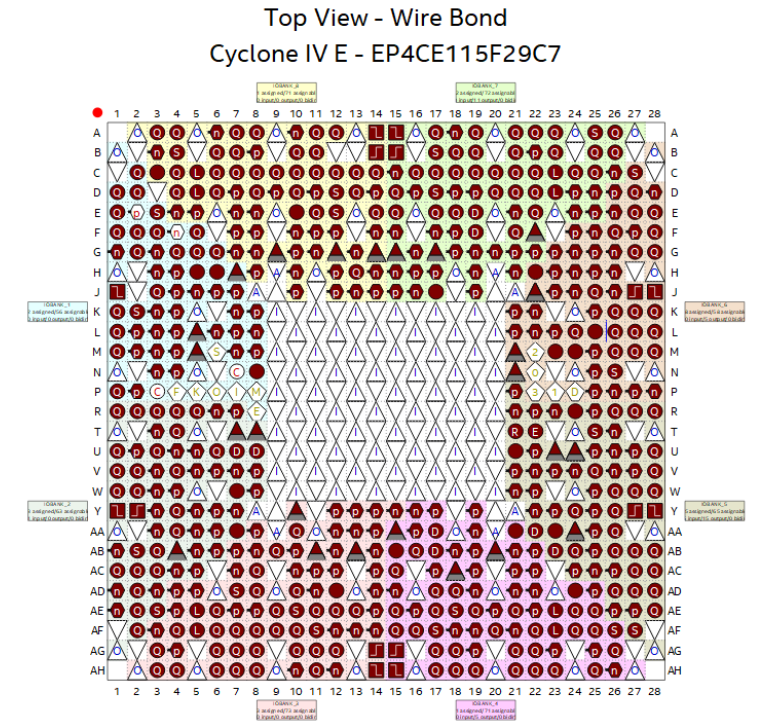
Named: *

Node Name	Direction	Location	I/O Bank	VREF Group	I/O Standard	Reserved
HEX0[6..0]	Output...				2.5 V	
HEX0[6]	Output	PIN_H22	6	B6_N0	2.5 V	
HEX0[5]	Output	PIN_J22	6	B6_N0	2.5 V	
HEX0[4]	Output	PIN_L25	6	B6_N1	2.5 V	
HEX0[3]	Output	PIN_L26	6	B6_N1	2.5 V	
HEX0[2]	Output	PIN_E17	7	B7_N2	2.5 V	
HEX0[1]	Output	PIN_F22	7	B7_N0	2.5 V	
HEX0[0]	Output	PIN_G18	7	B7_N2	2.5 V	
HEX1[6..0]	Output...				2.5 V	
HEX2[6..0]	Output...				2.5 V	
HEX3[6..0]	Output...				2.5 V ...fault)	
KEY[3..3]	Input Group				2.5 V	
KEY[3]	Input	PIN_R24	5	B5_N0	2.5 V	
LEDG[3..0]	Output...				2.5 V	
LEDG[3]	Output	PIN_E24	7	B7_N1	2.5 V	
LEDG[2]	Output	PIN_E25	7	B7_N1	2.5 V	
LEDG[1]	Output	PIN_E22	7	B7_N0	2.5 V	
LEDG[0]	Output	PIN_E21	7	B7_N0	2.5 V	
LEDR[3..0]	Output...				2.5 V	
LEDR[3]	Output	PIN_F21	7	B7_N0	2.5 V	
LEDR[2]	Output	PIN_E19	7	B7_N0	2.5 V	
LEDR[1]	Output	PIN_F19	7	B7_N0	2.5 V	
LEDR[0]	Output	PIN_G19	7	B7_N2	2.5 V	
SW[3..0]	Input Group				2.5 V	
SW[3]	Input	PIN_AD27	5	B5_N2	2.5 V	
SW[2]	Input	PIN_AC27	5	B5_N2	2.5 V	
SW[1]	Input	PIN_AC28	5	B5_N2	2.5 V	
SW[0]	Input	PIN_AR28	5	B5_N1	2.5 V	

Groups Report

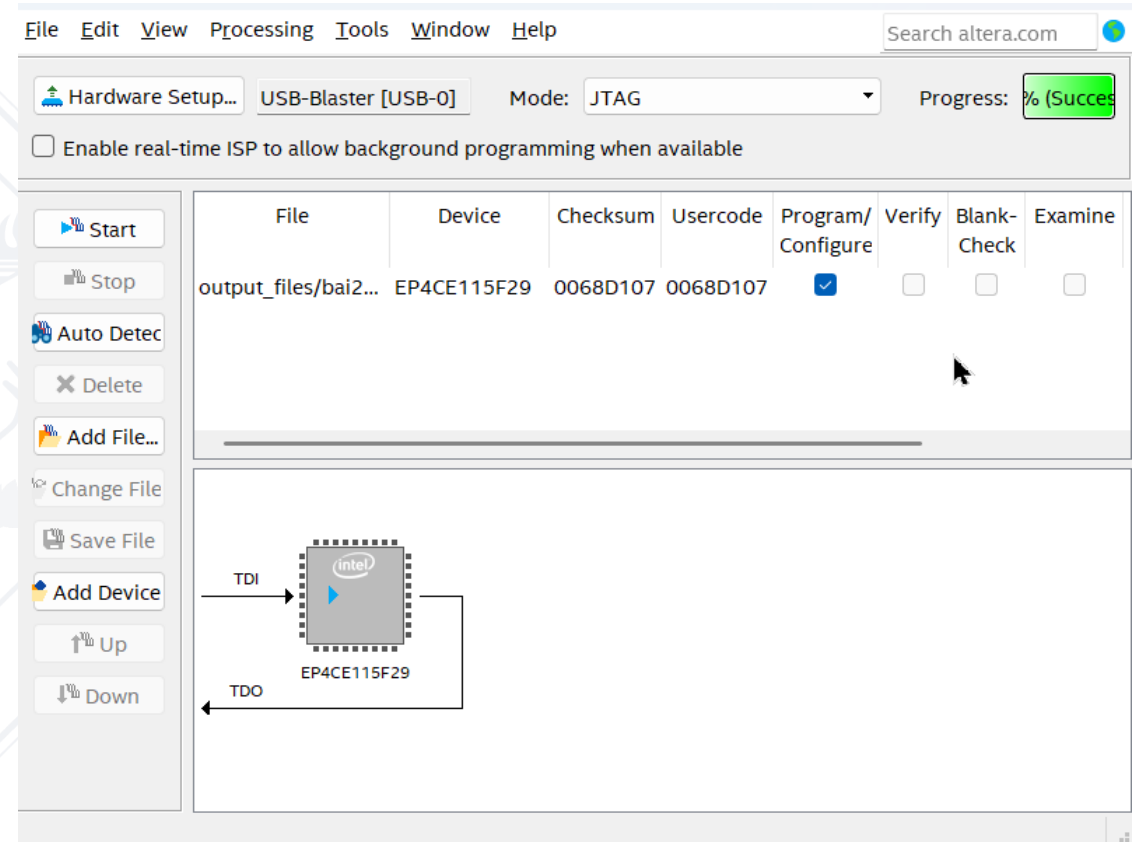
Named: * Edit: X HEX0[4]

Node Name	Direction	Location	I/O Bank	VREF Group	Pin Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair	Input Preservation
HEX3[5]	Output	PIN_AF23	4	B4_N0	PIN_AF23	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[4]	Output	PIN_AD24	4	B4_N0	PIN_AD24	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[3]	Output	PIN_AA21	4	B4_N0	PIN_AA21	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[2]	Output	PIN_AB20	4	B4_N0	PIN_AB20	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[1]	Output	PIN_U21	5	B5_N0	PIN_U21	2.5 V		8mA (default)	2 (default)		
HEX3[0]	Output	PIN_V21	5	B5_N1	PIN_V21	2.5 V		8mA (default)	2 (default)		
KEY[3]	Input	PIN_R24	5	B5_N0	PIN_R24	2.5 V		8mA (default)			



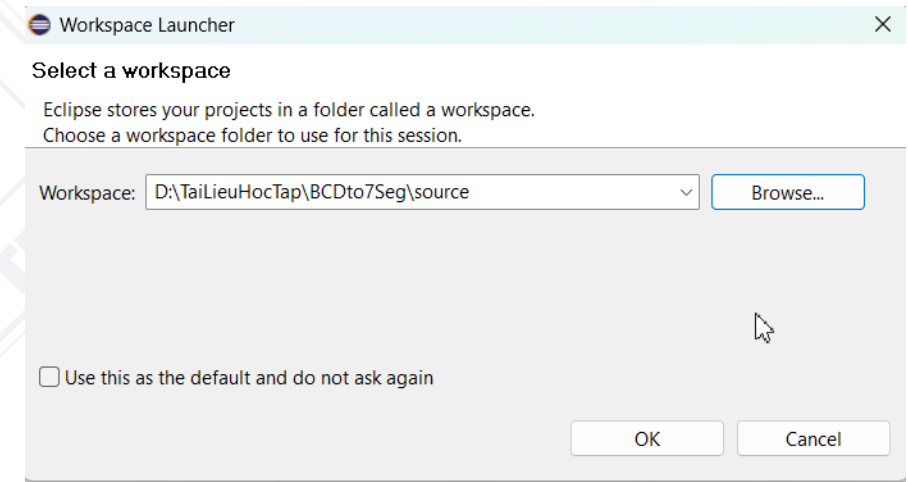
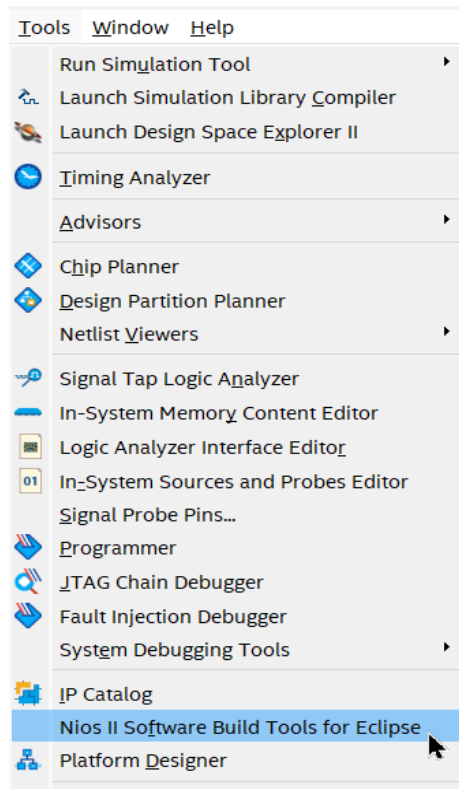
◆ 6. Nạp thiết kế cho chip FPGA

- Trên thanh công cụ vào *Tools - Programmer* để nạp thiết kế phần cứng của hệ thống nhúng.
- Trong cửa sổ *Programmer*, chọn *Add File* và chọn vào tập tin có đuôi *.sof* trong thư mục *output_files* của dự án.
- Tiếp theo vào *Hardware Setup* và chọn *USB-Blaster [USB-0]* và nhấn *Close*.
- Nhấn *Start* để tiến hành nạp thiết kế xuống chip FPGA. Nếu nạp thành công kết quả sẽ giống như hình bên.



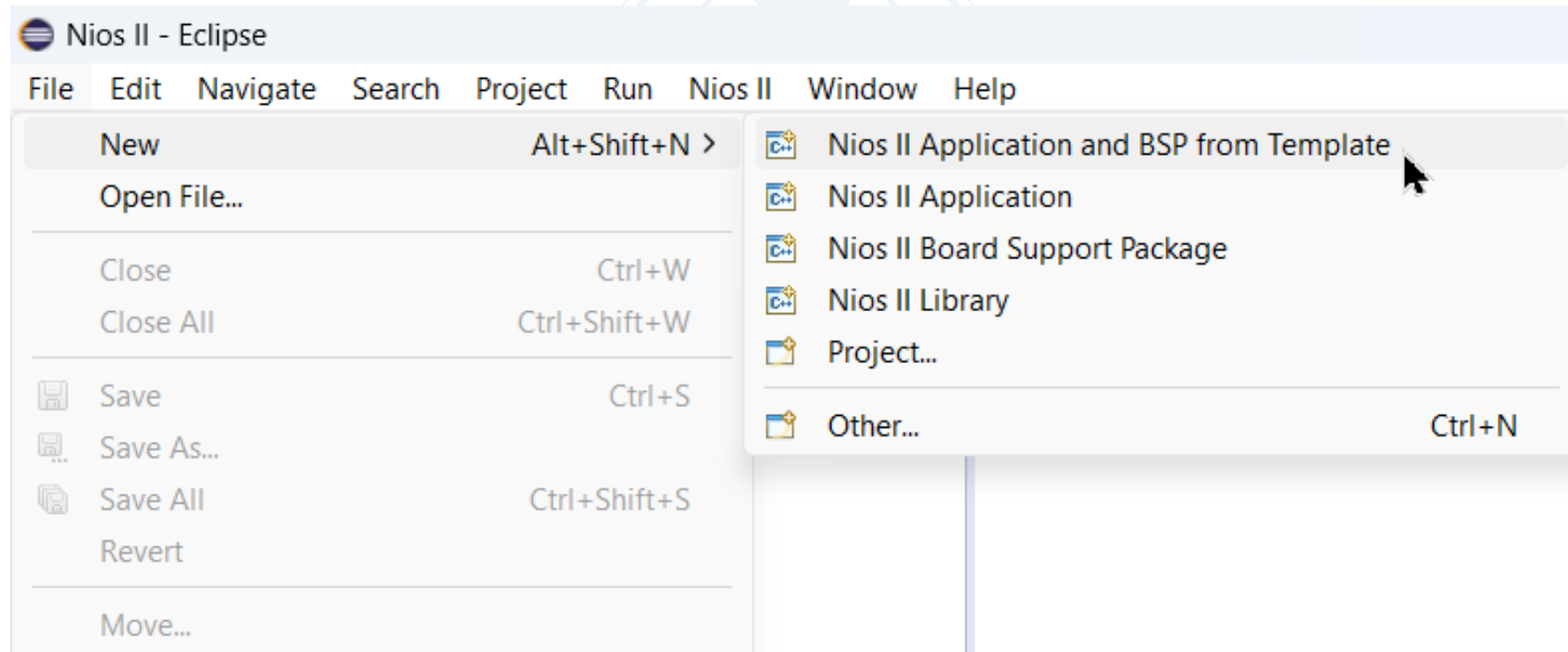
7. Lập trình phần mềm nhúng

- Để viết chương trình, ta sử dụng công cụ *Nios II Software Build Tools for Eclipse* bằng cách trên thanh công cụ vào *Tools – Nios II Software Build Tools for Eclipse*.
- Sau khi mở phần mềm, ta chọn đường dẫn để lưu trữ chương trình.



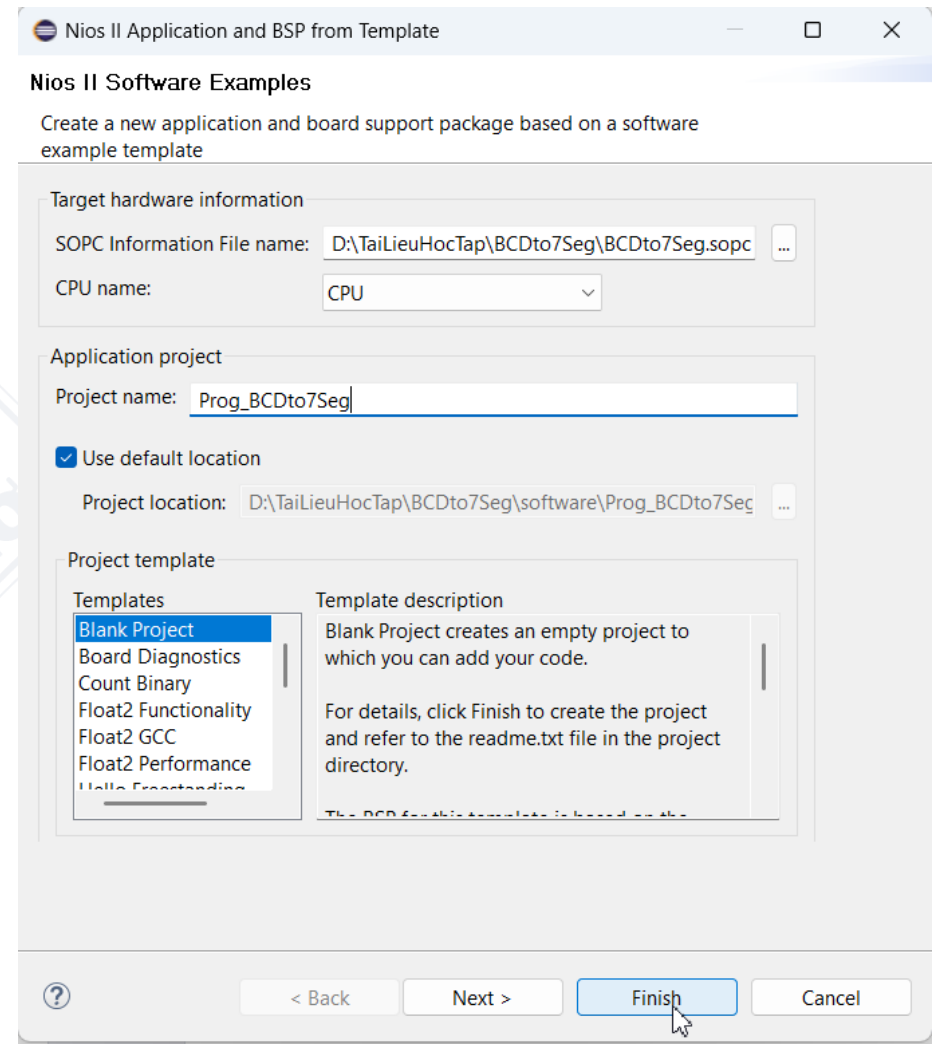
◆ 7. Lập trình phần mềm nhúng

- Tiếp theo trên thanh công cụ của *Nios II – Eclipse* vào *File – New – Nios II Application and BSP from Template* để tạo chương trình.



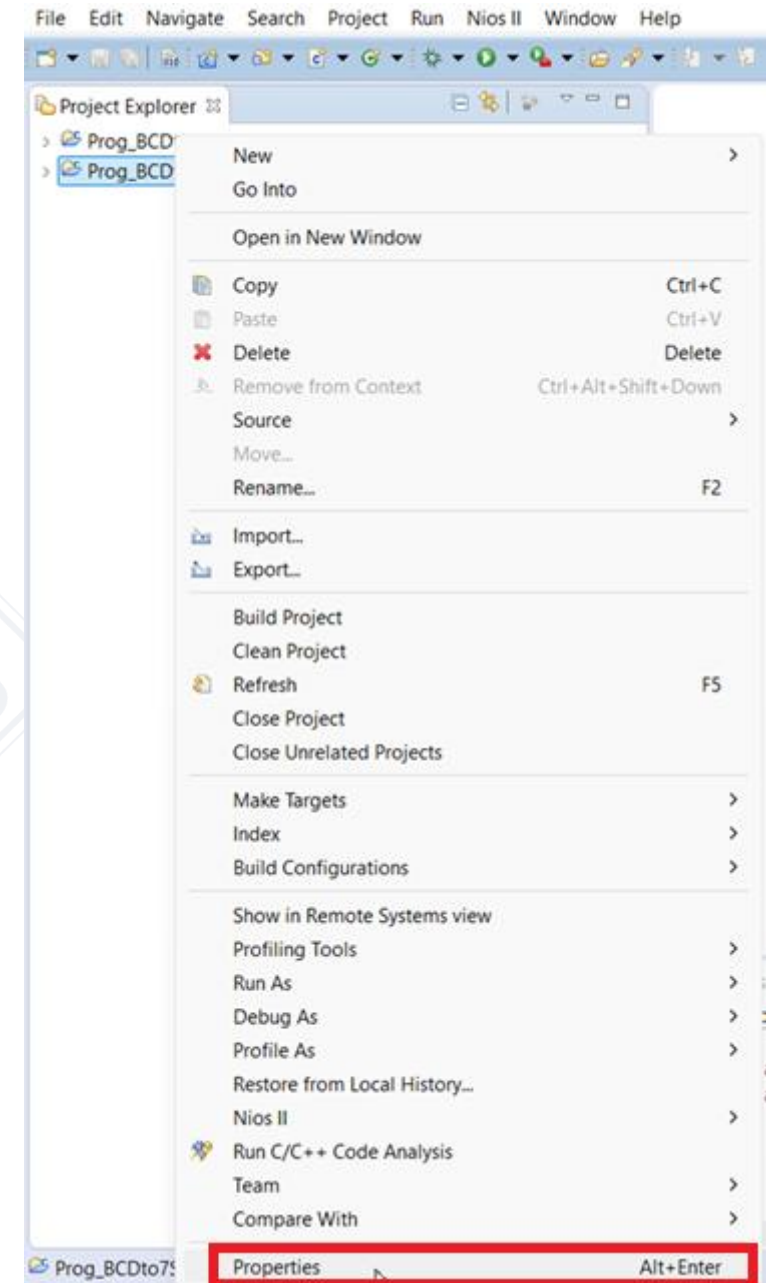
7. Lập trình phần mềm nhúng

- Ở mục *SOPC Information File name* ta chọn file *BCDto7Seg.sopcinfo* chứa thông tin của hệ thống.
- Ở *Project name*: ta đặt tên cho dự án.
- Ở phần *Templates*, ta chọn *Blank Project*.
- Sau đó nhấn *Finish*.



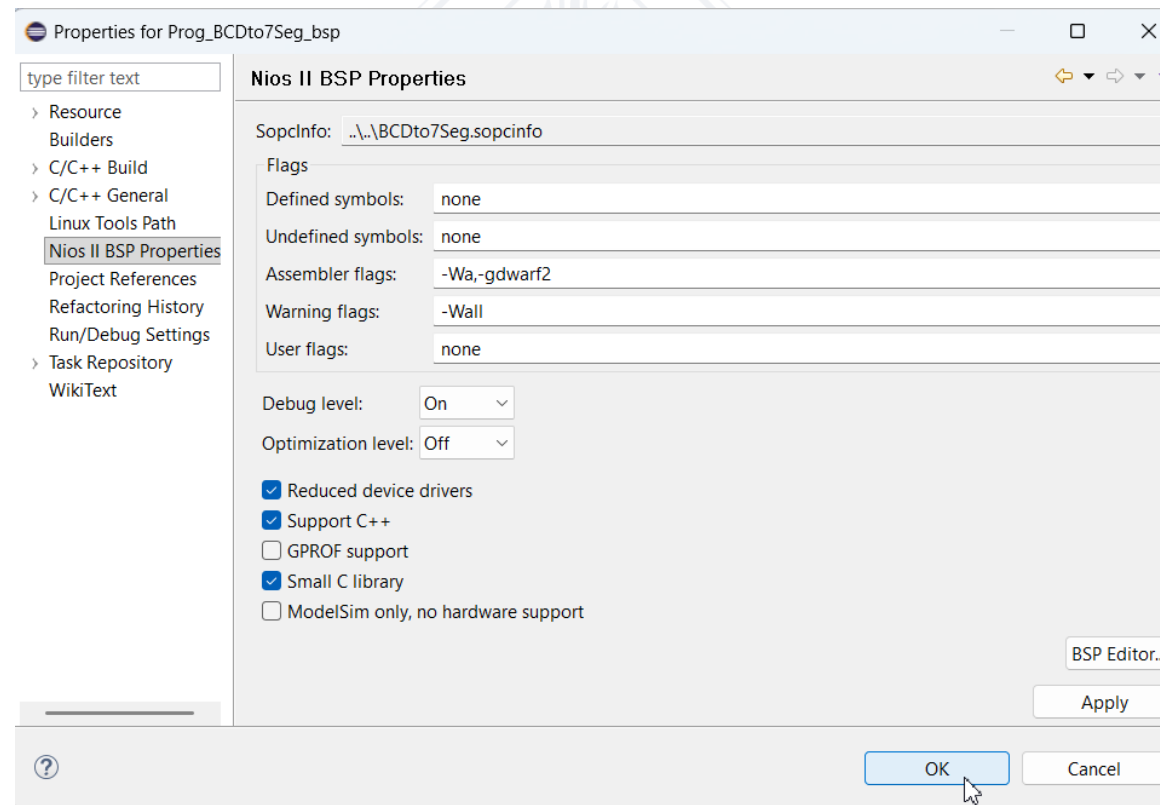
◆ 7. Lập trình phần mềm nhúng

- Trong cửa sổ *Project Explorer*, nhấp chọn *Prog_BCDto7Seg* và nhấn *Chuột phải* – chọn *Properties*.



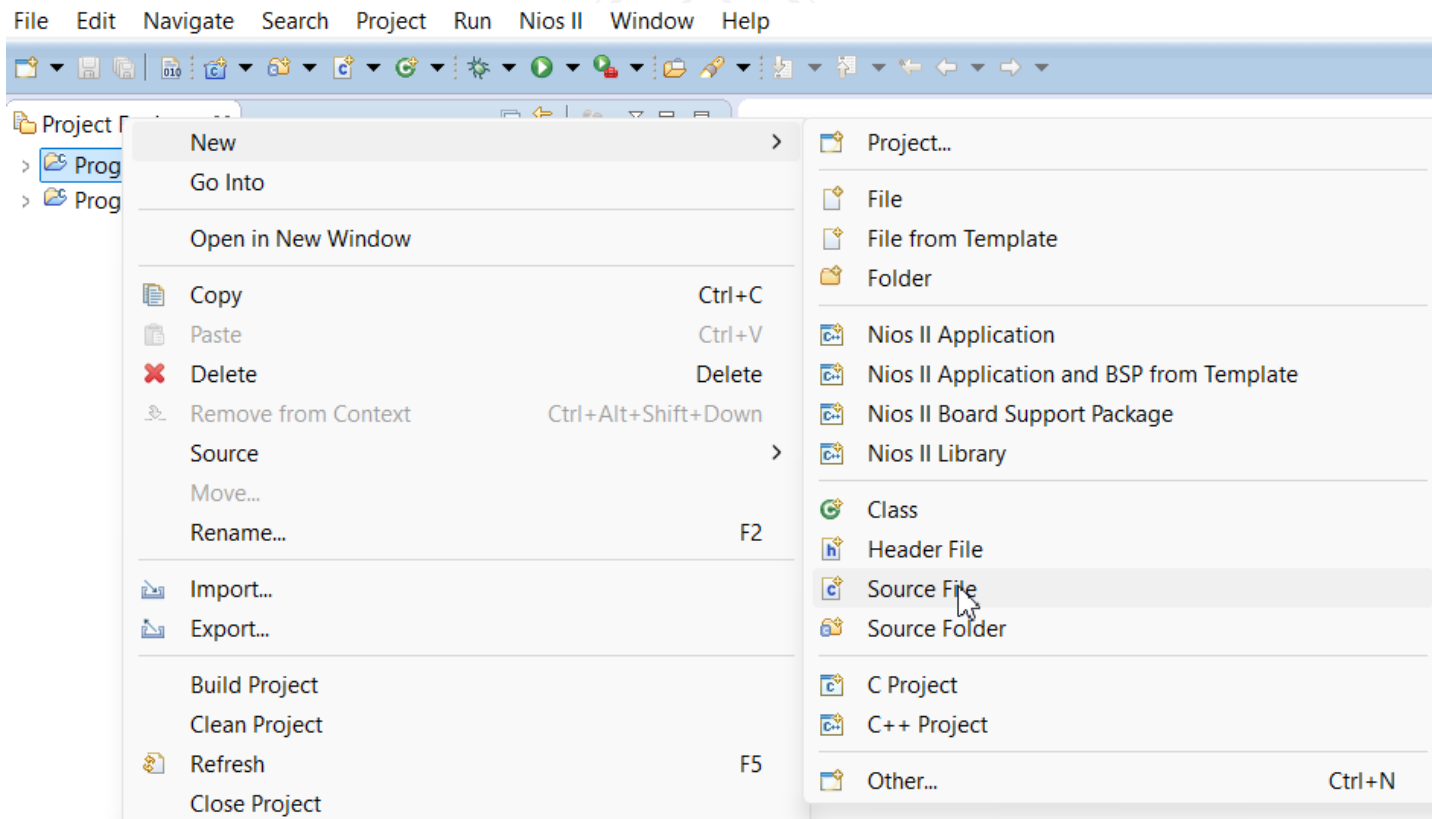
7. Lập trình phần mềm nhúng

- Trong cửa sổ *Properties* chọn *Nios II BSP Properties* và chọn vào các tùy chọn: *Reduced device drivers*, *Small C library* để chương trình có thể hoạt động với bộ nhớ on-chip dung lượng 32 KB. Sau đó chọn *Apply* - *OK*.



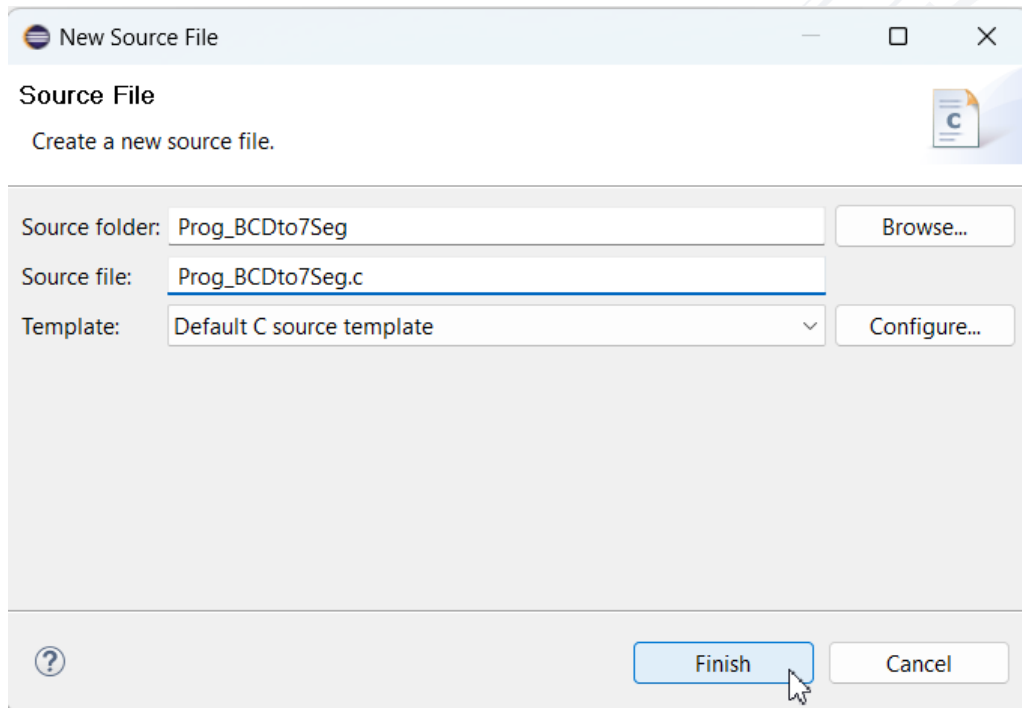
7. Lập trình phần mềm nhúng

- Để thêm tập tin chương trình vào dự án lập trình ta nhấp vào *Prog_BCDto7Seg* và *Click chuột phải* – chọn *New – Source File*.

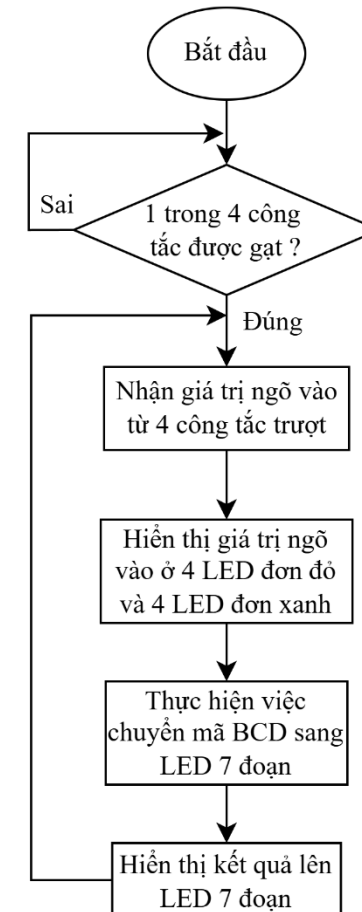


7. Lập trình phần mềm nhúng

- Ở cửa sổ *Source File*, đặt tên cho chương trình là *Prog_BCDto7Seg.c* và chọn *Template: Default C source template*



- Lưu đồ chương trình:



7. Lập trình phần mềm nhúng

- Code của chương trình như sau:

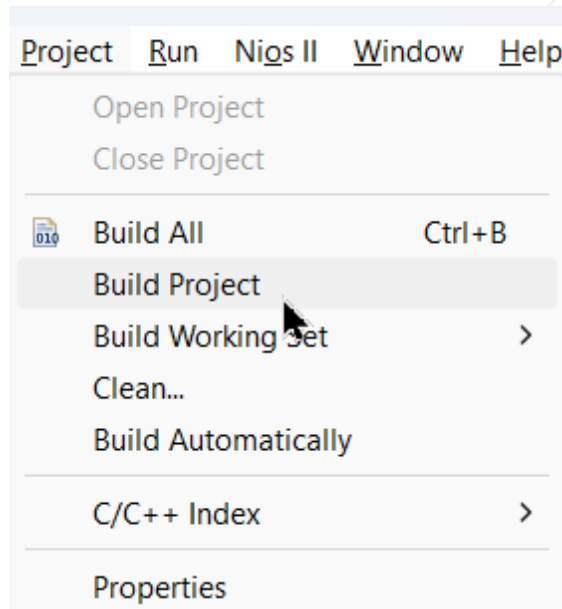
```
#include <system.h>
#include <unistd.h>
int main() {
    volatile int* SWS  = (int*) SWS_BASE;
    volatile int* LEDR  = (int*) LEDR_BASE;
    volatile int* LEDG  = (int*) LEDG_BASE;
    volatile int* HEX3_0 = (int*) HEX3_0_BASE;
    *LEDR = 0x00;
    *LEDG = 0x00;
    *HEX3_0 = 0x3f;
    while (((*SWS) & 0x0F) == 0);
    while(1) {
        *LEDR = *SWS;
        *LEDG = *SWS;
        switch (*SWS) {
            case 0x0: // 0000
                *HEX3_0 = 0x3f;
                break;
```

```
            case 0x1: // 0001
                *HEX3_0 = 0x06;
                break;
            case 0x2: // 0010
                *HEX3_0 = 0x5b;
                break;
            case 0x3: // 0011
                *HEX3_0 = 0x4f;
                break;
            case 0x4: // 0100
                *HEX3_0 = 0x66;
                break;
            case 0x5: // 0101
                *HEX3_0 = 0x6d;
                break;
            case 0x6: // 0110
                *HEX3_0 = 0x7d;
                break;
```

```
            case 0x7: // 0111
                *HEX3_0 = 0x07;
                break;
            case 0x8: // 1000
                *HEX3_0 = 0x7f;
                break;
            case 0x9: // 1001
                *HEX3_0 = 0x6f;
                break;
            default: // 1X1X
                *HEX3_0 = 0x00;
                break;
        }
    }
    return 0;
}
```

◆ 7. Lập trình phần mềm nhúng

- Sau khi viết xong nội dung chương trình, lưu lại *Ctrl + S* và tiến hành biên dịch bằng cách trên thanh công cụ vào *Project - Build Project*
- Nếu xuất hiện lỗi (error) hoặc cảnh báo (warning) trong quá trình biên dịch thì phải sửa cho đến khi biên dịch thành công. Kết quả khi biên dịch thành công.



Problems Tasks Console Properties

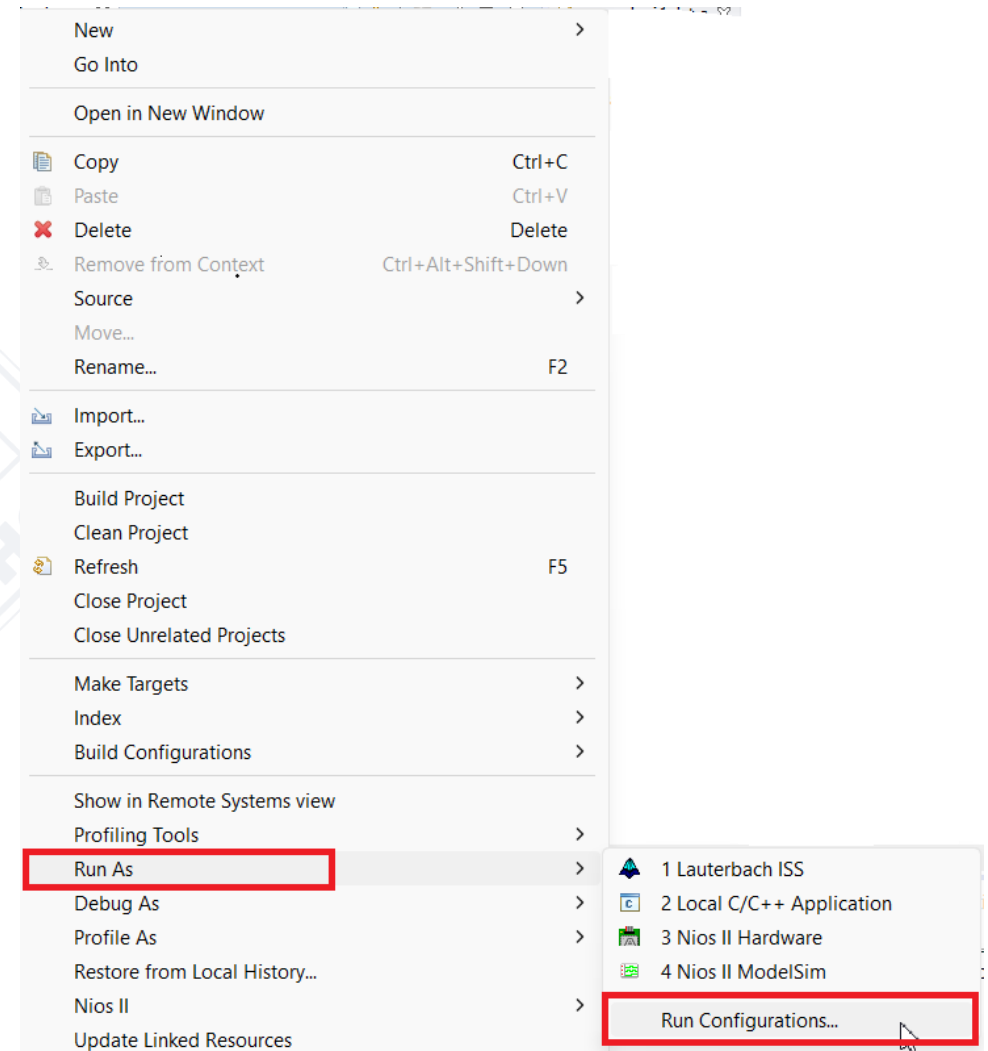
CDT Build Console [Prog_BCDto7Seg_bsp]

```
09:22:22 **** Incremental Build of configuration Nios II for project Prog_BCDto7Seg_bsp
make all
[BSP build complete]
```

```
09:22:22 Build Finished (took 286ms)
```

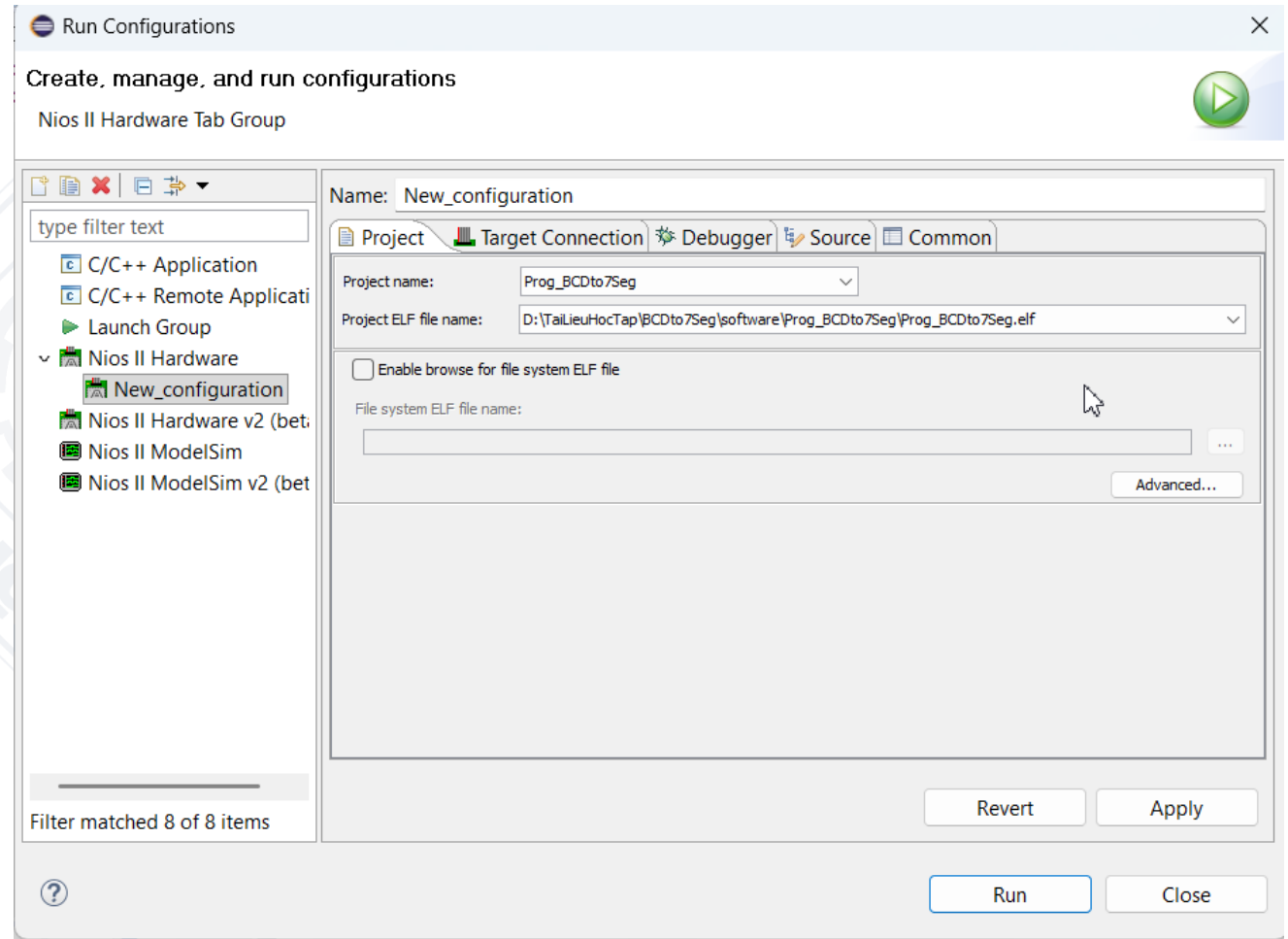
7. Lập trình phần mềm nhúng

- Để cấu hình cho việc thực thi chương trình trên hệ thống nhúng (board DE2-115), nhấp chọn thư mục *Prog_BCDto7Seg* ở cửa sổ *Project Explorer* tiếp theo Click chuột phải chọn *Run As - Run Configurations...*



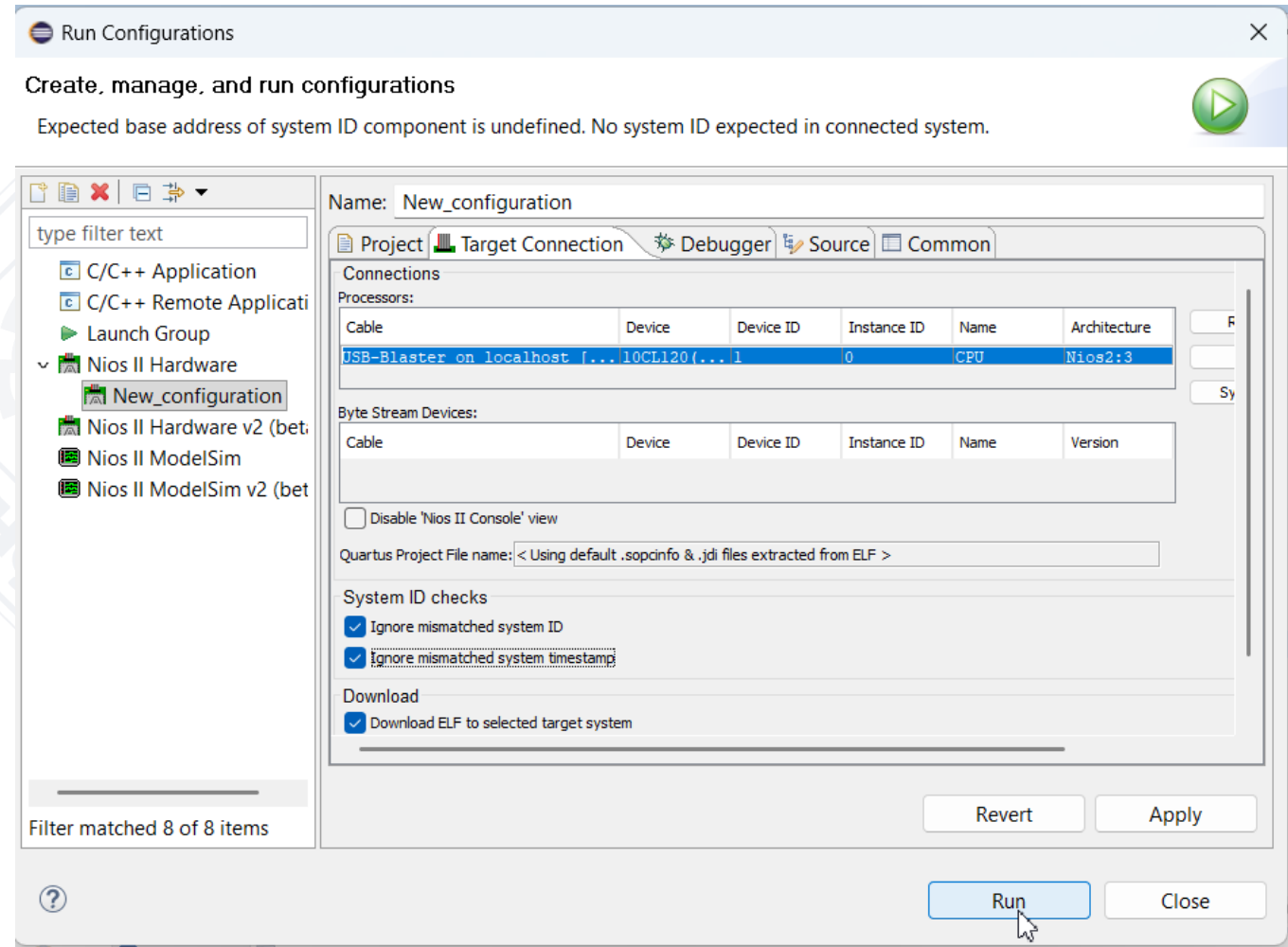
7. Lập trình phần mềm nhúng

- Click đúp chuột vào *Nios II Hardware* để tạo cấu hình mới.
- Cần kiểm tra lại tên dự án, tập tin đường dẫn đến file *.elf*.



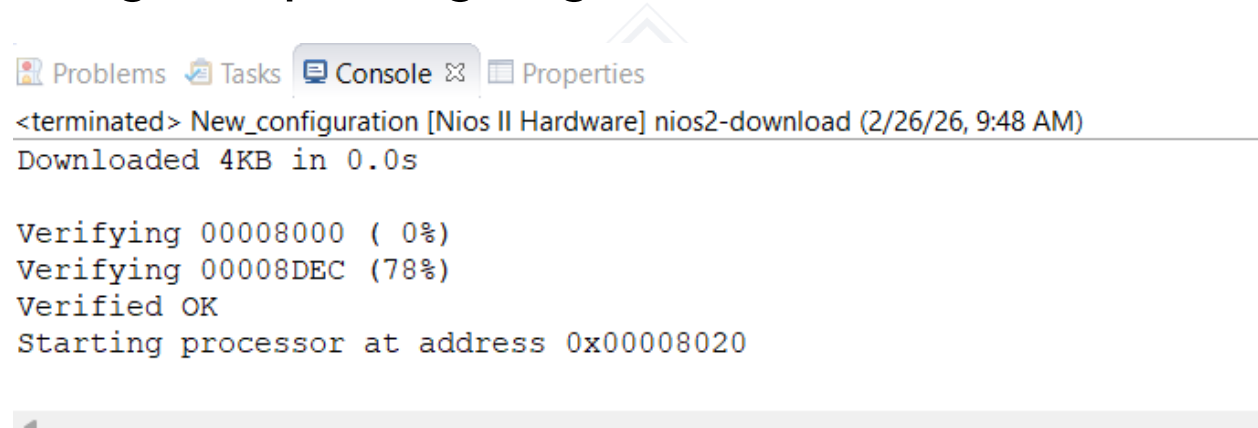
7. Lập trình phần mềm nhúng

- Nếu đúng thì chuyển qua tab *Target Connection*, nếu chưa thấy kết nối với *USB-Blaster on localhost* ở mục *Processor*. thì nhấn vào *Refresh Connections* để thiết lập lại kết nối.
- Ở mục *System ID checks* tick chọn vào các ô *Ignore mismatched system ID*, *Ignore mismatched system timestamp* để bỏ qua phần kiểm tra các thông tin này khi chạy chương trình.
- Sau đó nhấn *Apply - Run* để thực hiện chương trình.



◆ 7. Lập trình phần mềm nhúng

- Nếu nạp thành công kết quả sẽ giống như hình bên dưới.

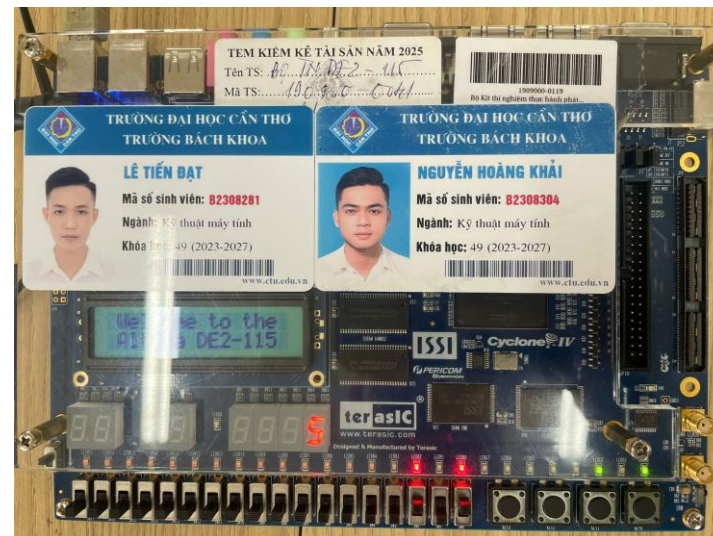
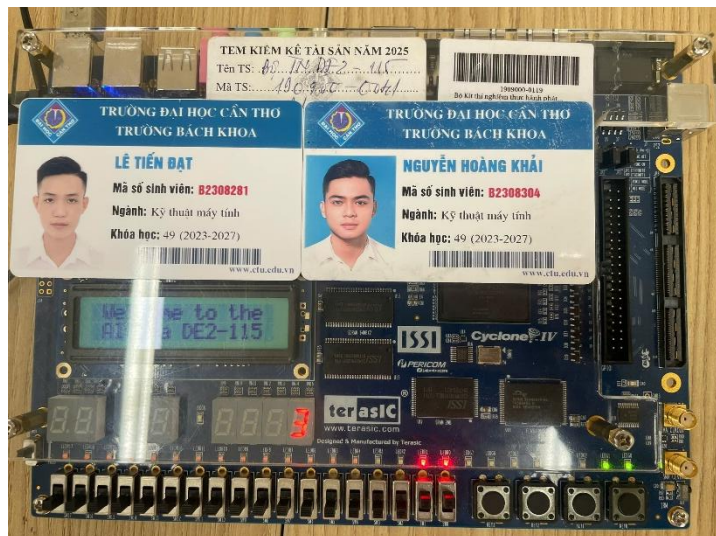
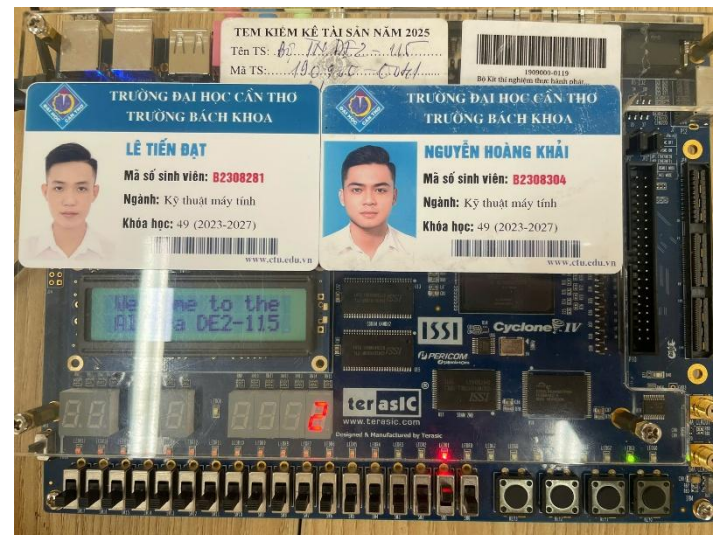
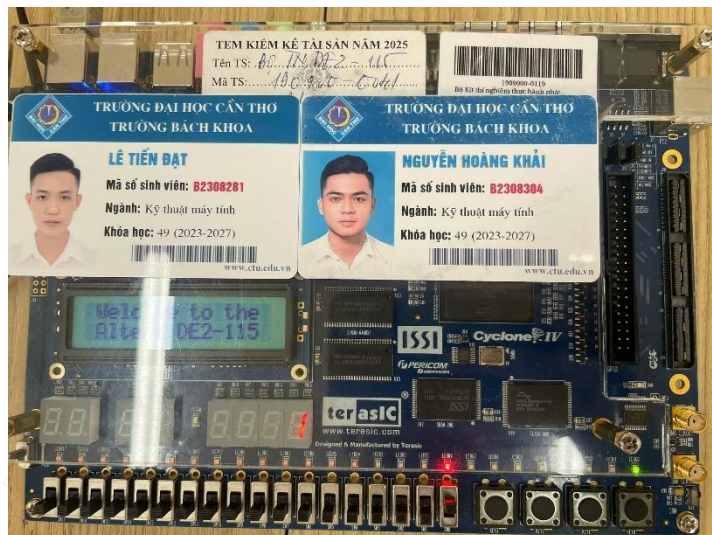
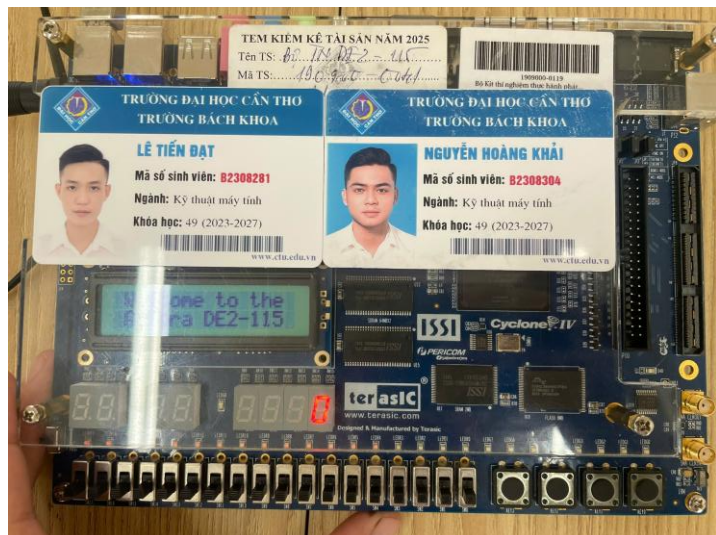


```
Problems Tasks Console Properties
<terminated> New_configuration [Nios II Hardware] nios2-download (2/26/26, 9:48 AM)
Downloaded 4KB in 0.0s

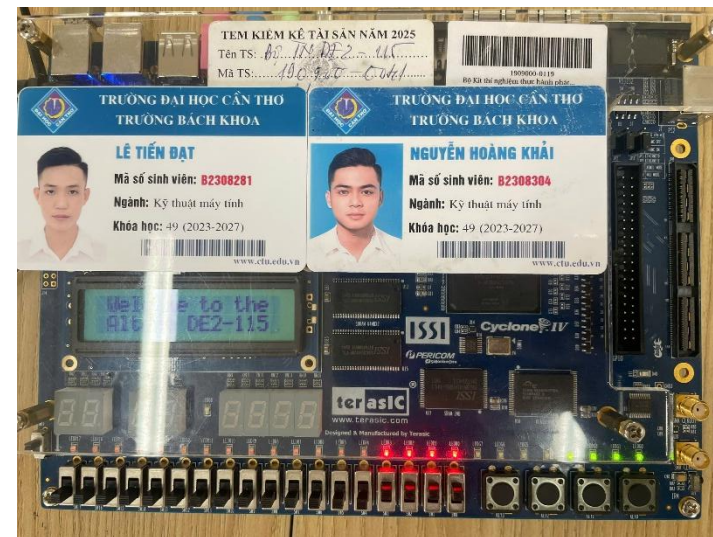
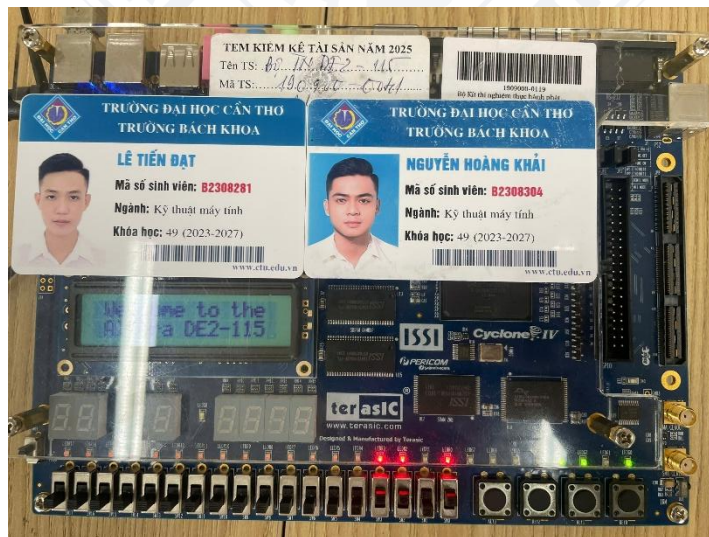
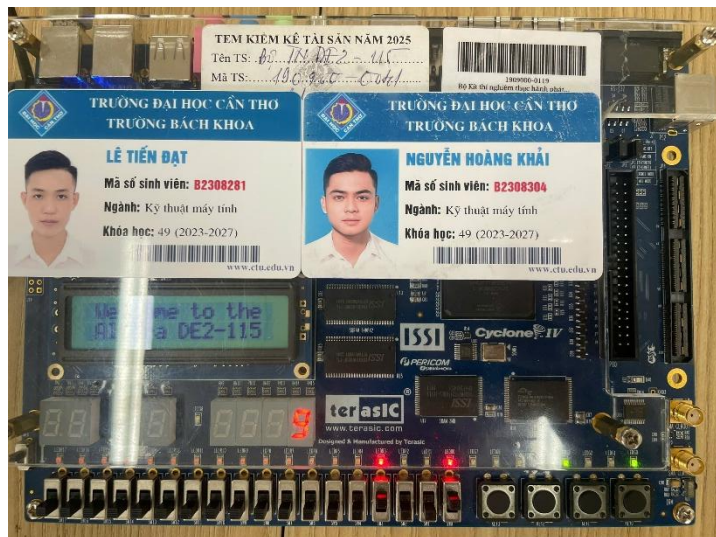
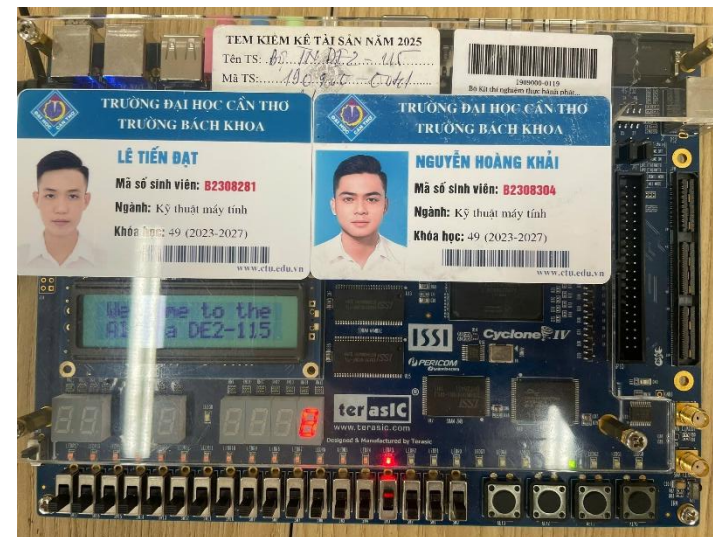
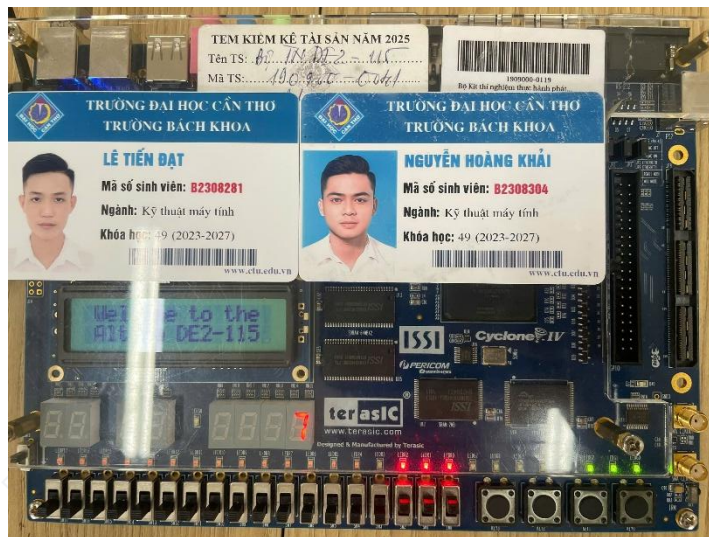
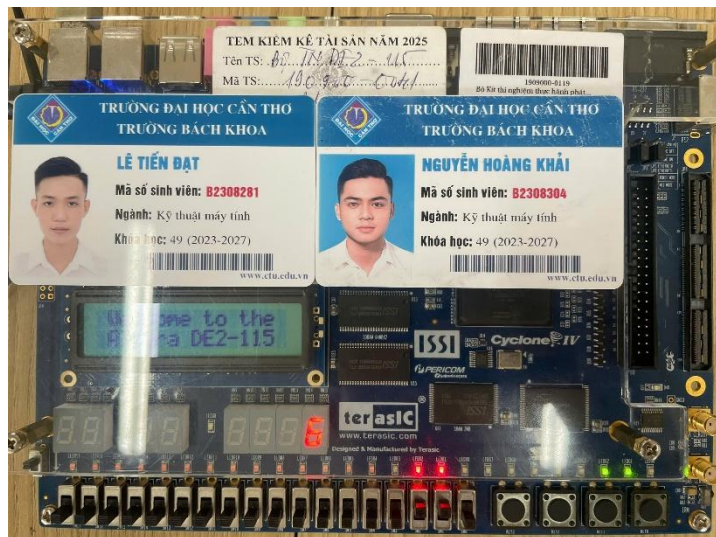
Verifying 00008000 ( 0%)
Verifying 00008DEC (78%)
Verified OK
Starting processor at address 0x00008020
```

- Nếu xuất hiện lỗi kit không nhận được file .e/f thì có thể chỉnh sửa cổng kết nối USB – Blaster và khởi động lại kit cho đến khi nạp thành công.

8. Kết quả thực thi chương trình



8. Kết quả thực thi chương trình



◆ 9. Nhận xét và kết luận

1. Nhận xét

- Hoàn thành đúng yêu cầu đề tài: Hệ thống đọc chuẩn xác mã BCD từ Switch, hiển thị trạng thái lên LED đơn và giải mã ra LED 7 đoạn.
- Hệ thống hoạt động ổn định, tốc độ đáp ứng nhanh, giao tiếp tốt giữa phần cứng và phần mềm.
- Hiểu rõ kiến trúc và quy trình thiết kế hệ thống nhúng trên FPGA.
- Thành thạo sử dụng các công cụ: Quartus Prime, Qsys (thiết kế phần cứng) và Eclipse (lập trình C/C++ cho Nios II).

◆ 9. Nhận xét và kết luận

2. Hướng phát triển

- Thêm 1 LED 7 đoạn để hiển thị giá trị lớn hơn 9.
- Tối ưu hóa phần mềm nhúng để tiết kiệm tài nguyên bộ nhớ và tốc độ xử lý của CPU (tối ưu code, vòng lặp, sử dụng mảng để lưu trạng thái LED 7 đoạn thay vì switch case).
- Thêm các phép toán cơ bản (cộng, trừ,...) trên mã BCD.
- Tích hợp hiển thị lên màn hình LCD 16x2 hoặc dùng thêm Nút nhấn để điều khiển.

 CẢM ƠN THẦY VÀ CÁC BẠN ĐÃ THEO DÕI